Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
    4.1. Diagram Color and Shape Legend
5. Architecture Intent View
    5.1. What This View Answers
    5.2. View-Scoped Projection
    5.3. System Boundary Diagram
    5.4. System Decomposition Diagram
    5.5. Functional Group Manhattan Matrix
    5.6. Functional Unit Dependency Diagram
    5.7. Functional Groups and Units (View Scoped)
6. Communication View
    6.1. What This View Answers
    6.2. View-Scoped Projection
    6.3. Coverage Gaps
    6.4. Recommended Next Evidence Additions
    6.5. Communication Path Graph
    6.6. Functional Groups and Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. View-Scoped Projection
    7.3. Coverage Gaps
    7.4. Recommended Next Evidence Additions
    7.5. Deployment Artifact Relationship Graph
    7.6. Platform Operations Graph
    7.7. Functional Groups and Units (View Scoped)
8. Security View
    8.1. What This View Answers
    8.2. View-Scoped Projection
    8.3. Coverage Gaps
    8.4. Recommended Next Evidence Additions
    8.5. Attack Vector Chapters
9. Traceability View
    9.1. What This View Answers
    9.2. View-Scoped Projection
    9.3. Coverage Gaps
    9.4. Recommended Next Evidence Additions
    9.5. Requirement-to-Unit Mapping (Compact)
    9.6. Verification Result Mapping
10. Traceability Appendix

10.1. Requirement Details
10.2. Verification Inventory
11. Reference Index
11.1. Authored References
11.2. Catalog References
11.3. Inferred Runtime References
11.4. Inferred Code References
11.5. Verification References

---

1. Introduction

This document describes a GitHub App that reviews pull requests using AWS Bedrock and Lambda. The authored functional architecture remains stable while deployment/runtime/code evidence is inferred from infrastructure manifests, source files, and verification artifacts.

1.1. Scope and Assumptions

- `Functional Groups` and `Functional Units` are the stable authored design backbone.
- `Runtime elements` and `code elements` are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- `Requirements` and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`)	stable	Authored capability boundaries and ownership responsibilities are stable for this release.	7	7	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.
Communication View (`VIEW-COMMUNICATION`)	in-review	Communication sequencing is defined, while protocol-level metadata is still being expanded.	7	3	4	low	Counts a unit as covered when direct inferred runtime evidence exists.
Deployment View (`VIEW-DEPLOYMENT`)	in-review	Deployment and runtime mapping are present; observability and ownership evidence are still evolving.	7	3	4	low	Counts a unit as covered when direct inferred runtime evidence exists.
Security View (`VIEW-SECURITY`)	draft	Core threats are modeled; control-depth and detection mappings are still being expanded.	7	7	2	high	Counts a unit as covered when explicit authored attack-vector mapping exists.
Traceability View (`VIEW-TRACEABILITY`)	in-review	Requirement mappings exist; evidence completeness and consistency are still being hardened.	7	7	4	high	Counts a unit as covered when direct inferred runtime or code evidence exists.

4. Terms and Definitions

Term	Definition
`actor` (EM-ACTOR)	A person or operational role that interacts with functional units.
`ai review is enabled` (FEAT-AI-REVIEW)	Feature flag enabling model-assisted review. Aliases:ai review enabled
`attack vector` (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
`audit logging is enabled` (FEAT-AUDIT-LOGGING)	Feature flag enabling audit metadata persistence. Aliases:audit enabled
`authored mapping` (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
`aws sdk for go` (REF-AWS-SDK-GO)	SDK dependency used for AWS runtime integrations. Aliases:aws sdk
`aws secrets manager` (REF-AWS-SECRETS-MANAGER)	Managed secret store for application credentials and config values. Aliases:secrets manager
`bedrock runtime` (REF-AWS-BEDROCK-RUNTIME)	External Bedrock runtime endpoint used for model inference. Aliases:bedrock api
`bedrock runtime is unavailable` (EVT-BEDROCK-UNAVAILABLE)	Event raised when Bedrock inference path is degraded. Aliases:bedrock unavailable
`check run summary` (DATA-CHECKRUN-SUMMARY)	Aggregated review result posted as a GitHub check run. Aliases:check run summaries
`code element` (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
`developer` (ACT-DEVELOPER)	Engineer opening and updating pull requests. Aliases:pr author
`diff context` (DATA-DIFF-CONTEXT)	Pull request file changes and metadata prepared for analysis. Aliases:normalized review context, pull request files, pull request metadata, review context, review-ready context
`diff context is prepared` (EVT-CONTEXT-PREPARED)	Event indicating context assembly completed and is ready for analysis. Aliases:context prepared
`functional group` (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
`functional unit` (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.

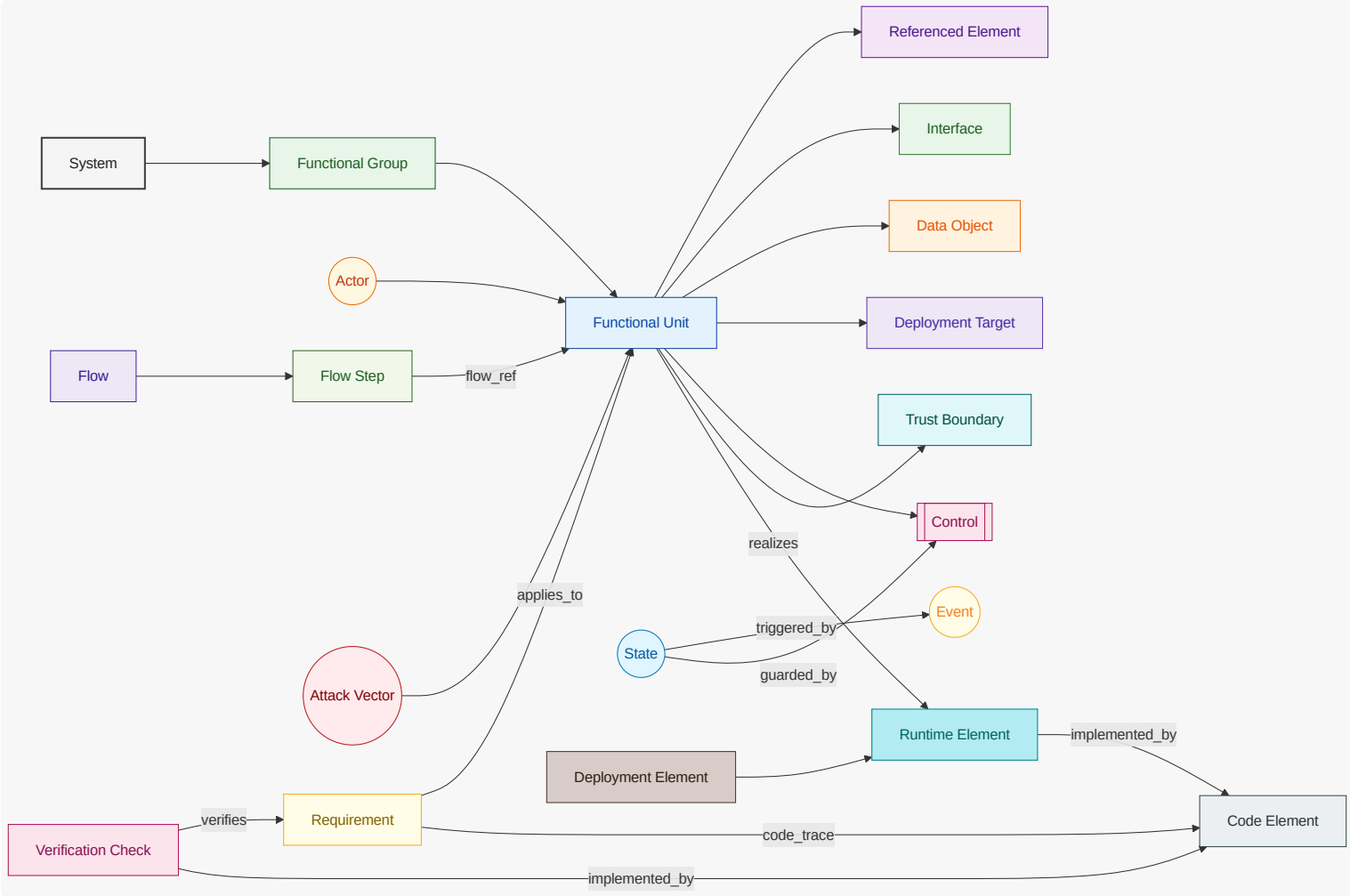
Term	Definition
`github app api` (REF-GITHUB-APP-API)	External GitHub API used for pull request metadata and review publication. Aliases:github api
`github rest sdk` (REF-GITHUB-REST-SDK)	SDK dependency used for GitHub App REST interactions. Aliases:github sdk
`github webhook ingress` (FU-GITHUB-WEBHOOK-INGRESS)	Validates webhook signatures and routes trusted pull request events. Aliases:webhook ingress
`inferred mapping` (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
`lambda runtime operations` (FU-LAMBDA-RUNTIME-OPERATIONS)	Manages runtime deployment and operational controls. Aliases:lambda ops
`normal mode is active` (MODE-NORMAL)	Default operating mode using AI plus deterministic checks. Aliases:normal mode
`platform operations` (FG-PLATFORM-OPERATIONS)	Functional group for deployment lifecycle and secret/config controls. Aliases:runtime operations
`platform operator` (ACT-PLATFORM-OPERATOR)	Owner of deployment/runtime operations and reliability controls. Aliases:platform ops
`policy checks` (FU-POLICY-CHECKS)	Evaluates deterministic repository policy checks. Aliases:deterministic checks
`policy checks are enabled` (FEAT-POLICY-CHECKS)	Feature flag enabling deterministic policy checks. Aliases:policy checks enabled
`policy ruleset` (REF-POLICY-RULESET)	Deterministic review policy bundle used during analysis. Aliases:policy bundle
`pr context assembly` (FU-PR-CONTEXT-ASSEMBLY)	Fetches changed files and prepares bounded review context payloads. Aliases:context assembly
`pr integration` (FG-PR-INTEGRATION)	Functional group for webhook ingress and pull request context intake. Aliases:github integration
`pr review system` (SYS-PR-REVIEW-SYSTEM)	GitHub App review system-of-interest that ingests PR events and publishes review outcomes. Aliases:the pr review system
`prompt injection in diff` (AV-PROMPT-INJECTION-IN-DIFF)	Malicious prompt-like content embedded in pull request code changes. Aliases:prompt injection
`pull request opened event is received` (EVT-PR-OPENED)	Event raised when a new pull request is opened. Aliases:pr opened, webhook events

Term	Definition
`pull request review is active` (STATE-PR-REVIEW-ACTIVE)	State where review processing is currently active for a pull request. Aliases:review active
`pull request spam abuse` (AV-PR-SPAM-ABUSE)	High-volume event abuse causing review capacity exhaustion. Aliases:pr spam
`pull request synchronize event is received` (EVT-PR-SYNCHRONIZE)	Event raised when new commits are pushed to an open pull request. Aliases:authorized events, pr synchronize
`redacted audit metadata` (DATA-REDACTED-AUDIT-METADATA)	Sanitized prompt and finding metadata retained for audit.
`referenced element` (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
`repository maintainer` (ACT-REPOSITORY-MAINTAINER)	Owner of merge decisions and repository quality gates. Aliases:maintainer
`requirement` (EM-REQUIREMENT)	A structured requirement statement that defines expected system behavior and maps to functional ownership.
`result metadata` (DATA-RESULT-METADATA)	Metadata describing review output characteristics for audit and reporting.
`review findings are generated` (EVT-FINDINGS-GENERATED)	Event indicating analysis output is ready for publication. Aliases:findings generated, review outcome
`review findings are published` (EVT-FINDINGS-PUBLISHED)	Event indicating review output has been published to GitHub checks/comments. Aliases:findings published, publication output
`review intelligence` (FG-REVIEW-INTELLIGENCE)	Functional group for AI and deterministic review analysis and publication. Aliases:review system
`review orchestration` (FU-REVIEW-ORCHESTRATION)	Coordinates Bedrock analysis with deterministic checks and fallback logic. Aliases:orchestration
`review publication` (FU-REVIEW-PUBLICATION)	Publishes review findings to GitHub checks and comments. Aliases:publication
`review request rate limit is exceeded` (EVT-RATE-LIMIT-EXCEEDED)	Event raised when PR review rate guardrails are hit. Aliases:rate limit exceeded
`runtime element` (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
`secret leakage in review output` (AV-SECRET-LEAKAGE-IN-REVIEW)	Unintended credential disclosure in generated review findings. Aliases:secret leakage

Term	Definition
`secrets and configuration` (FU-SECRETS-CONFIGURATION)	Manages credentials and secure runtime configuration. Aliases:secret config
`security engineer` (ACT-SECURITY-ENGINEER)	Owner of security policy checks and finding triage. Aliases:security reviewer
`spoofed github webhook` (AV-SPOOFED-GITHUB-WEBHOOK)	Forged webhook request attempting unauthorized review execution. Aliases:webhook spoofing
`upward linking` (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.
`verification check` (EM-VERIFICATION-CHECK)	An inferred or authored verification artifact that validates requirement behavior with test evidence.
`webhook signature` (DATA-WEBHOOK-SIGNATURE)	Signature used to verify webhook authenticity.
`webhook signature is valid` (COND-WEBHOOK-SIGNATURE-VALID)	Condition indicating webhook HMAC verification succeeded. Aliases:signature valid

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



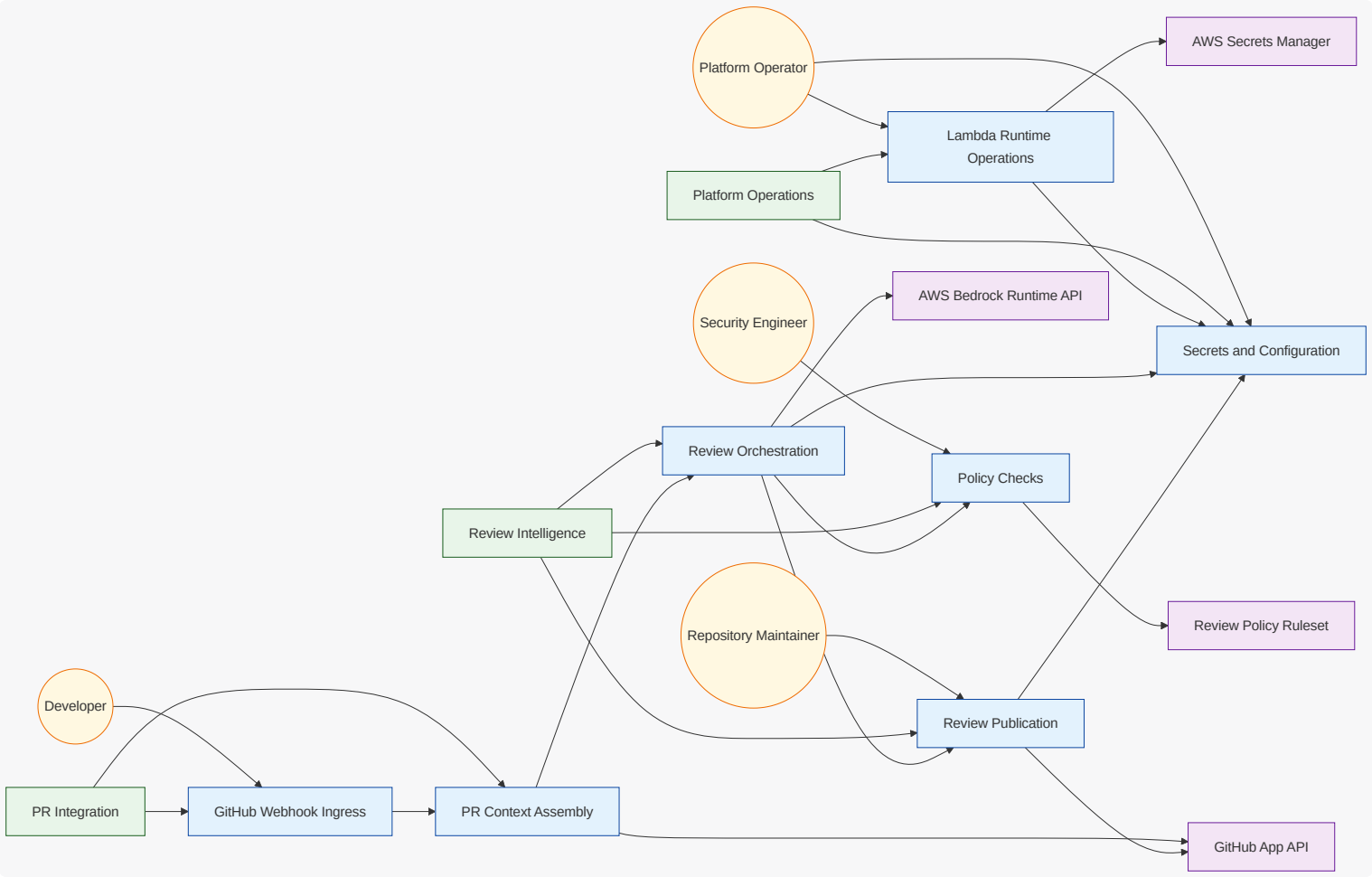
5. Architecture Intent View

Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

5.1. What This View Answers

- What are the main capability areas?
- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

5.2. View-Scoped Projection



Node ID	Kind	Label
ACT-DEVELOPER	actor	Developer
ACT-PLATFORM-OPERATOR	actor	Platform Operator
ACT-REPOSITORY-MAINTAINER	actor	Repository Maintainer
ACT-SECURITY-ENGINEER	actor	Security Engineer
FG-PLATFORM-OPERATIONS	functional_group	Platform Operations
FG-PR-INTEGRATION	functional_group	PR Integration
FG-REVIEW-INTELLIGENCE	functional_group	Review Intelligence

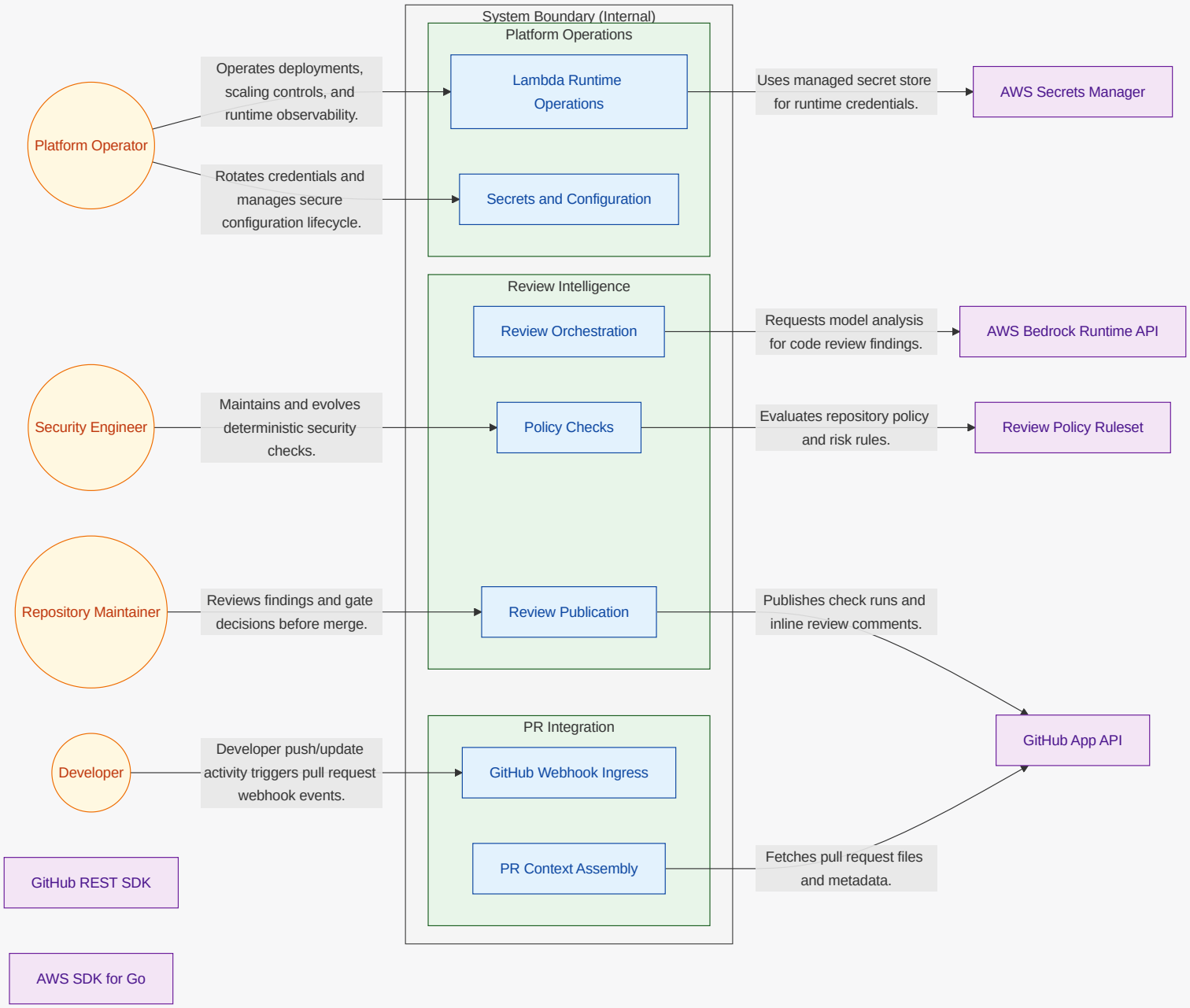
Node ID	Kind	Label
`FU-GITHUB-WEBHOOK-INGRESS`	functional_unit	`GitHub Webhook Ingress`
`FU-LAMBDA-RUNTIME-OPERATIONS`	functional_unit	`Lambda Runtime Operations`
`FU-POLICY-CHECKS`	functional_unit	`Policy Checks`
`FU-PR-CONTEXT-ASSEMBLY`	functional_unit	`PR Context Assembly`
`FU-REVIEW-ORCHESTRATION`	functional_unit	`Review Orchestration`
`FU-REVIEW-PUBLICATION`	functional_unit	`Review Publication`
`FU-SECRETS-CONFIGURATION`	functional_unit	`Secrets and Configuration`
`REF-AWS-BEDROCK-RUNTIME`	referenced_element	AWS `Bedrock Runtime` API
`REF-AWS-SECRETS-MANAGER`	referenced_element	`AWS Secrets Manager`
`REF-GITHUB-APP-API`	referenced_element	`GitHub App API`
`REF-POLICY-RULESET`	referenced_element	Review `Policy Ruleset`

Mapping Type	From	To
contains	`FG-PLATFORM-OPERATIONS`	`FU-LAMBDA-RUNTIME-OPERATIONS`
contains	`FG-PLATFORM-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
contains	`FG-PR-INTEGRATION`	`FU-GITHUB-WEBHOOK-INGRESS`
contains	`FG-PR-INTEGRATION`	`FU-PR-CONTEXT-ASSEMBLY`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-POLICY-CHECKS`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-REVIEW-ORCHESTRATION`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-REVIEW-PUBLICATION`

Mapping Type	From	To
depends_on	FU-GITHUB-WEBHOOK-INGRESS	FU-PR-CONTEXT-ASSEMBLY
depends_on	FU-LAMBDA-RUNTIME-OPERATIONS	FU-SECRETS-CONFIGURATION
depends_on	FU-LAMBDA-RUNTIME-OPERATIONS	REF-AWS-SECRETS-MANAGER
depends_on	FU-POLICY-CHECKS	REF-POLICY-RULESET
depends_on	FU-PR-CONTEXT-ASSEMBLY	FU-REVIEW-ORCHESTRATION
depends_on	FU-PR-CONTEXT-ASSEMBLY	REF-GITHUB-APP-API
depends_on	FU-REVIEW-ORCHESTRATION	FU-POLICY-CHECKS
depends_on	FU-REVIEW-ORCHESTRATION	FU-REVIEW-PUBLICATION
depends_on	FU-REVIEW-ORCHESTRATION	FU-SECRETS-CONFIGURATION
depends_on	FU-REVIEW-ORCHESTRATION	REF-AWS-BEDROCK-RUNTIME
depends_on	FU-REVIEW-PUBLICATION	FU-SECRETS-CONFIGURATION
depends_on	FU-REVIEW-PUBLICATION	REF-GITHUB-APP-API
interacts_with	ACT-DEVELOPER	FU-GITHUB-WEBHOOK-INGRESS
interacts_with	ACT-PLATFORM-OPERATOR	FU-LAMBDA-RUNTIME-OPERATIONS
interacts_with	ACT-PLATFORM-OPERATOR	FU-SECRETS-CONFIGURATION
interacts_with	ACT-REPOSITORY-MAINTAINER	FU-REVIEW-PUBLICATION
interacts_with	ACT-SECURITY-ENGINEER	FU-POLICY-CHECKS

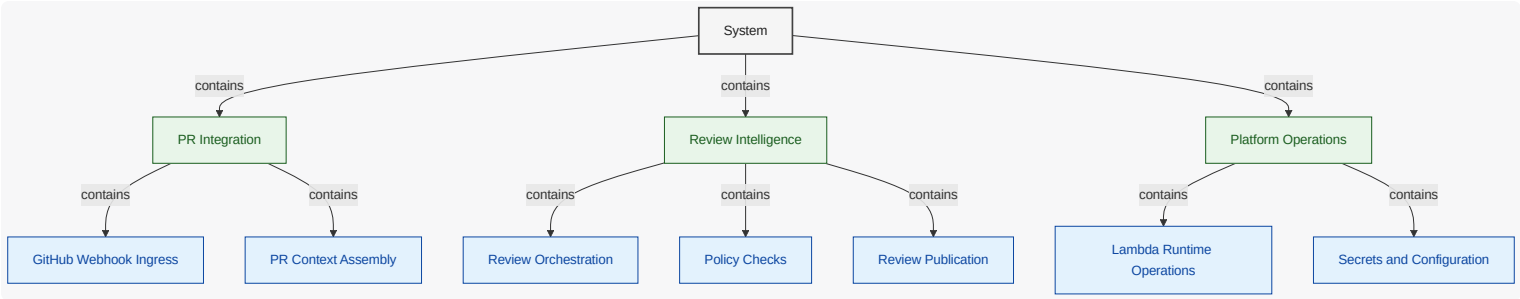
5.3. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.


System Boundary (Internal)
Platform Operations
Lambda Runtime Operations
Secrets and Configuration
Review Intelligence
Review Orchestration
Policy Checks
Review Publication
PR Integration
GitHub Webhook Ingress
PR Context Assembly
Platform Operator
Operates deployments, scaling controls, and runtime observability.
Rotates credentials and manages secure configuration lifecycle.
Uses managed secret store for runtime credentials.
AWS Secrets Manager
Requests model analysis for code review findings.
AWS Bedrock Runtime API
Security Engineer
Maintains and evolves deterministic security checks.
Evaluates repository policy and risk rules.
Review Policy Ruleset
Repository Maintainer
Reviews findings and gate decisions before merge.
Publishes check runs and inline review comments.
GitHub App API
Developer
Developer push/update activity triggers pull request webhook events.
Fetches pull request files and metadata.
GitHub REST SDK
AWS SDK for Go

5.4. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.



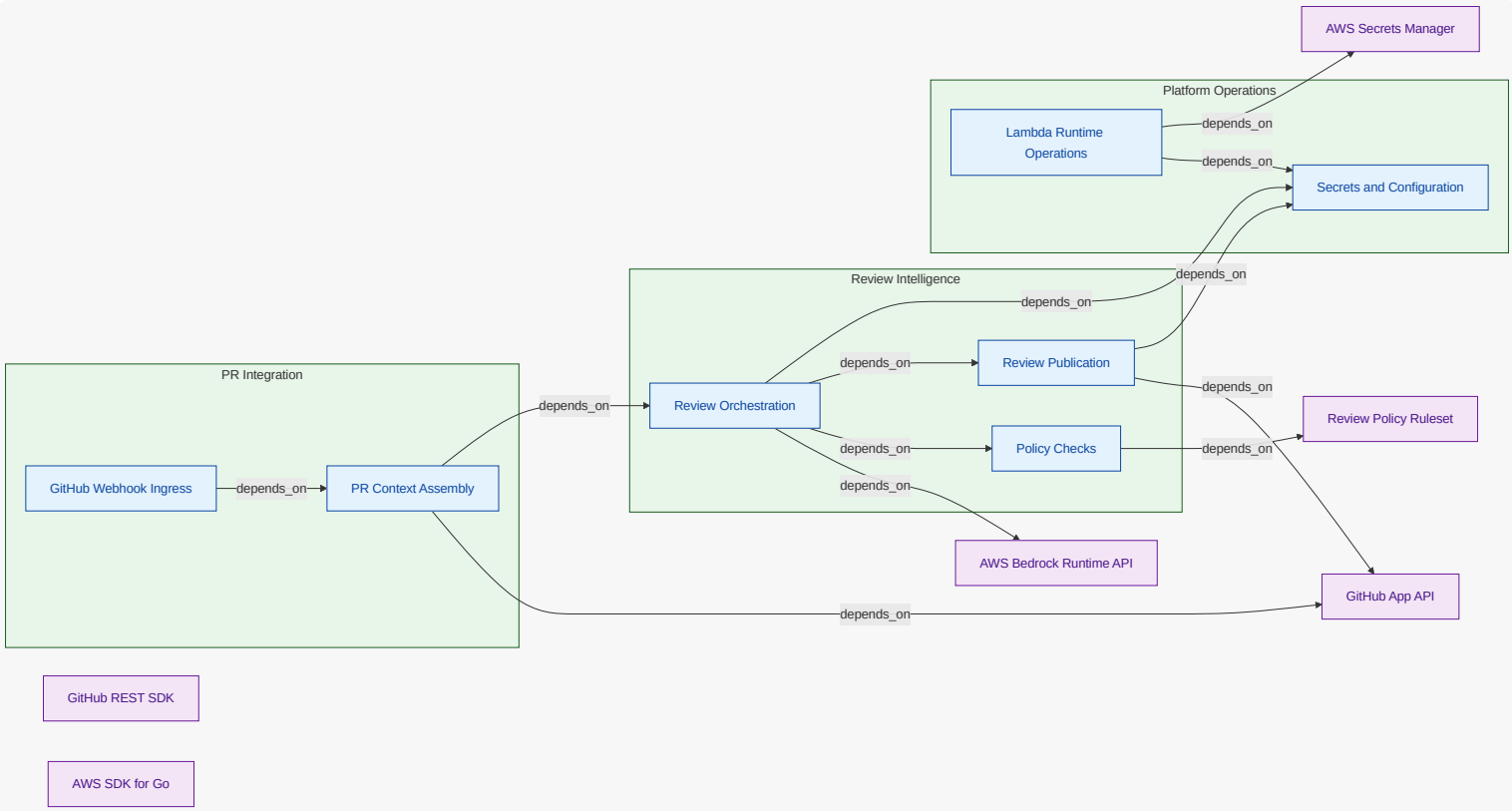
5.5. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

	Policy Checks	
GitHub Webhook Ingress	Review Orchestration	Lambda Runtime Operations
PR Context Assembly	Review Publication	Secrets and Configuration
PR Integration	Review Intelligence	Platform Operations

5.6. Functional Unit Dependency Diagram

What this diagram shows: dependency paths between functional units (grouped in Functional Group containers) and key referenced elements. Arrow meaning: `A -→|depends_on| B` means unit `A` requires behavior or data from `B` to fulfill its responsibility.



5.7. Functional Groups and Units (View Scoped)

5.7.1. PR Integration (FG-PR-INTEGRATION)

Ingress and `context assembly` for GitHub pull request events.

`PR Integration` owns webhook trust boundaries and pull request context intake. It converts incoming GitHub events into normalized review work with clear boundaries.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Verifies `webhook signatures`, validates event type, and starts review workflow routing.

Owned Responsibility

functional responsibility for `github webhook ingress`; includes decision and `orchestration` flow to `PR Context Assembly`

Key Collaborators

`PR Context Assembly`

Linked Requirements

`REQ-PRR-001`, `REQ-PRR-008`, `REQ-PRR-010`

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Fetches changed files and diff metadata and prepares bounded `review context` payloads.

Owned Responsibility

functional responsibility for `pr context assembly`; includes decision and `orchestration` flow to `GitHub App API` and decision and `orchestration` flow to `Review Orchestration`

Key Collaborators

`GitHub App API`, `Review Orchestration`

Linked Requirements

`REQ-PRR-002`

5.7.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

AI and deterministic review analysis with result `publication`.

`Review Intelligence` owns generated findings, `deterministic checks`, and final `publication` behavior. It combines Bedrock reasoning with predictable `policy checks` and emits auditable review outputs.

Policy Checks (FU-POLICY-CHECKS)

Runs `deterministic checks` for secrets, risky IaC changes, and repository policy constraints.

Owned Responsibility

functional responsibility for `policy checks`; includes decision and `orchestration` flow to Review `Policy Ruleset`

Key Collaborators

Review `Policy Ruleset`

Linked Requirements

`REQ-PRR-004`, `REQ-PRR-006`

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Calls Bedrock models, merges AI and deterministic findings, and handles graceful degradation paths.

Owned Responsibility

functional responsibility for `review orchestration`; includes decision and `orchestration` flow to AWS `Bedrock Runtime` API and decision and `orchestration` flow to `Policy Checks`

Key Collaborators

AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration`

Linked Requirements

`REQ-PRR-003`, `REQ-PRR-006`

Review Publication (FU-REVIEW-PUBLICATION)

Publishes check-run summaries and inline comments back to GitHub pull requests.

Owned Responsibility

functional responsibility for `review publication`; includes decision and `orchestration` flow to `GitHub App API` and decision and `orchestration` flow to `Secrets and Configuration`

Key Collaborators

`GitHub App API`, `Secrets and Configuration`

Linked Requirements

`REQ-PRR-005`, `REQ-PRR-006`, `REQ-PRR-007`, `REQ-PRR-008`, `REQ-PRR-009`, `REQ-PRR-010`

5.7.3. Platform Operations (FG-PLATFORM-OPERATIONS)

Lambda deployment lifecycle, `runtime operations`, and secrets/configuration.

`Platform Operations` owns safe deployment, runtime controls, and secret/config lifecycle for the app. It enables dependable review operation without owning business-level review policy semantics.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Manages Lambda packaging/deployment lifecycle and operational health guardrails.

Owned Responsibility

functional responsibility for `lambda runtime operations`; includes decision and `orchestration` flow to `AWS Secrets Manager` and decision and `orchestration` flow to `Secrets and Configuration`

Key Collaborators

`AWS Secrets Manager`, `Secrets and Configuration`

Linked Requirements

no explicit authored `requirement` trigger

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Manages GitHub App credentials, Bedrock config, and redaction-safe runtime configuration.

Owned Responsibility

functional responsibility for `secrets and configuration`

Key Collaborators

none

Linked Requirements

`REQ-PRR-007`

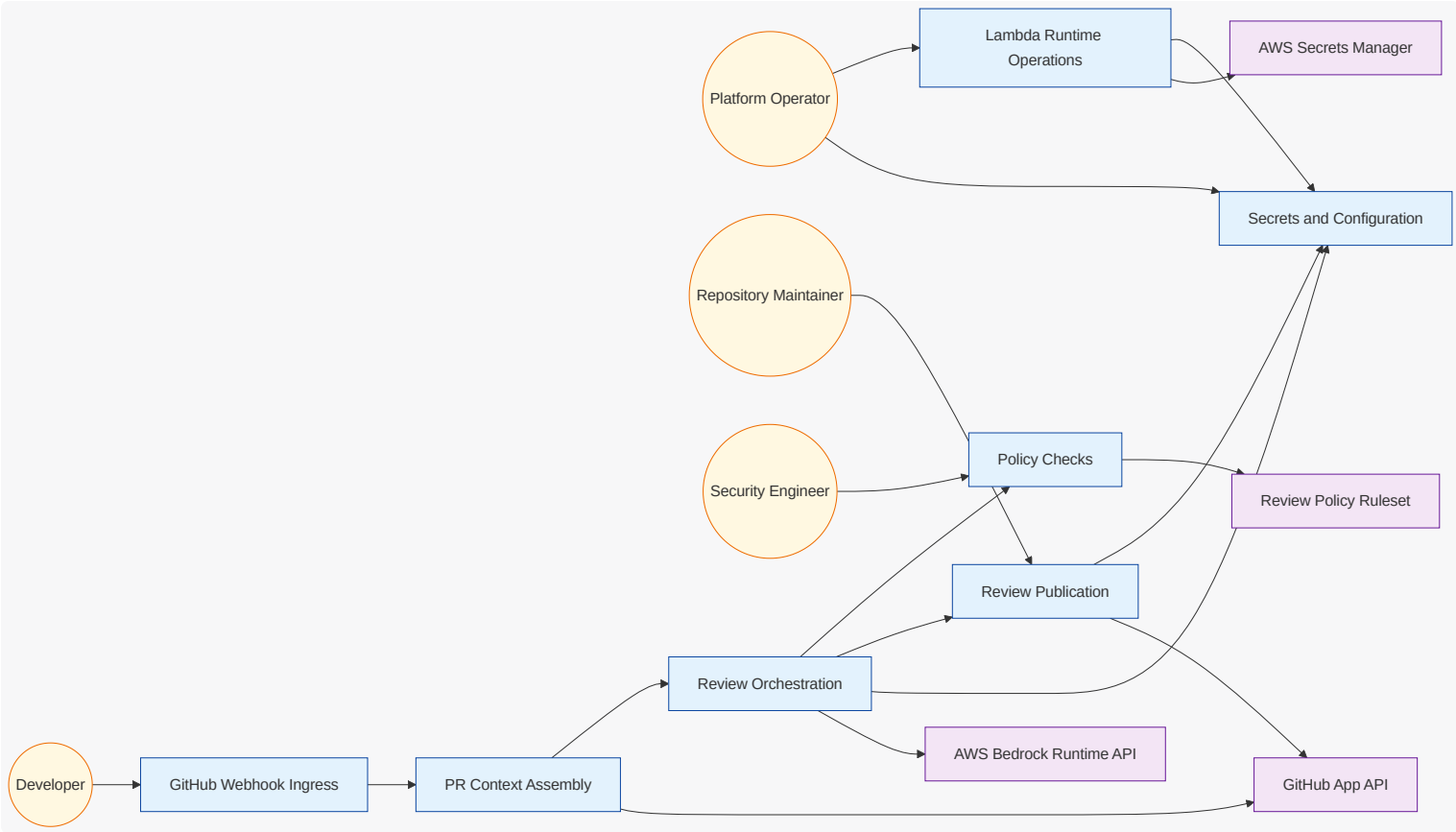
6. Communication View

Show who communicates with whom across key request paths, including interface edges and dependency handoffs.

6.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request flow?

6.2. View-Scoped Projection



Node ID	Kind	Label
ACT-DEVELOPER	actor	Developer
ACT-PLATFORM-OPERATOR	actor	Platform Operator
ACT-REPOSITORY-MAINTAINER	actor	Repository Maintainer
ACT-SECURITY-ENGINEER	actor	Security Engineer

Node ID	Kind	Label
`FU-GITHUB-WEBHOOK-INGRESS`	functional_unit	`GitHub Webhook Ingress`
`FU-LAMBDA-RUNTIME-OPERATIONS`	functional_unit	`Lambda Runtime Operations`
`FU-POLICY-CHECKS`	functional_unit	`Policy Checks`
`FU-PR-CONTEXT-ASSEMBLY`	functional_unit	`PR Context Assembly`
`FU-REVIEW-ORCHESTRATION`	functional_unit	`Review Orchestration`
`FU-REVIEW-PUBLICATION`	functional_unit	`Review Publication`
`FU-SECRETS-CONFIGURATION`	functional_unit	`Secrets and Configuration`
`REF-AWS-BEDROCK-RUNTIME`	referenced_element	AWS `Bedrock Runtime` API
`REF-AWS-SECRETS-MANAGER`	referenced_element	`AWS Secrets Manager`
`REF-GITHUB-APP-API`	referenced_element	`GitHub App API`
`REF-POLICY-RULESET`	referenced_element	Review `Policy Ruleset`

Mapping Type	From	To
depends_on	`FU-GITHUB-WEBHOOK-INGRESS`	`FU-PR-CONTEXT-ASSEMBLY`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`REF-AWS-SECRETS-MANAGER`
depends_on	`FU-POLICY-CHECKS`	`REF-POLICY-RULESET`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`FU-REVIEW-ORCHESTRATION`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`REF-GITHUB-APP-API`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-POLICY-CHECKS`

Mapping Type	From	To
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-REVIEW-PUBLICATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`REF-AWS-BEDROCK-RUNTIME`
depends_on	`FU-REVIEW-PUBLICATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-PUBLICATION`	`REF-GITHUB-APP-API`
interacts_with	`ACT-DEVELOPER`	`FU-GITHUB-WEBHOOK-INGRESS`
interacts_with	`ACT-PLATFORM-OPERATOR`	`FU-LAMBDA-RUNTIME-OPERATIONS`
interacts_with	`ACT-PLATFORM-OPERATOR`	`FU-SECRETS-CONFIGURATION`
interacts_with	`ACT-REPOSITORY-MAINTAINER`	`FU-REVIEW-PUBLICATION`
interacts_with	`ACT-SECURITY-ENGINEER`	`FU-POLICY-CHECKS`

6.3. Coverage Gaps

Coverage summary: 3/7 units have direct evidence coverage in this view.

- `Lambda Runtime Operations` has no direct inferred runtime/deployment evidence yet.
- `PR Context Assembly` has no direct inferred runtime/deployment evidence yet.
- `Policy Checks` has no direct inferred runtime/deployment evidence yet.
- `Secrets and Configuration` has no direct inferred runtime/deployment evidence yet.

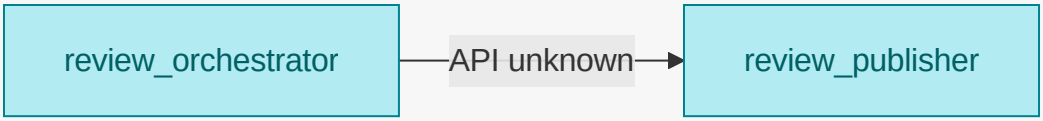
6.4. Recommended Next Evidence Additions

- Add explicit communication metadata (protocol, sync/async/event/callback) to `authored mappings` where currently implicit.
- Expose service protocol/port metadata in Helm or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

6.5. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

> **NOTE** This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior.



6.6. Functional Groups and Units (View Scoped)

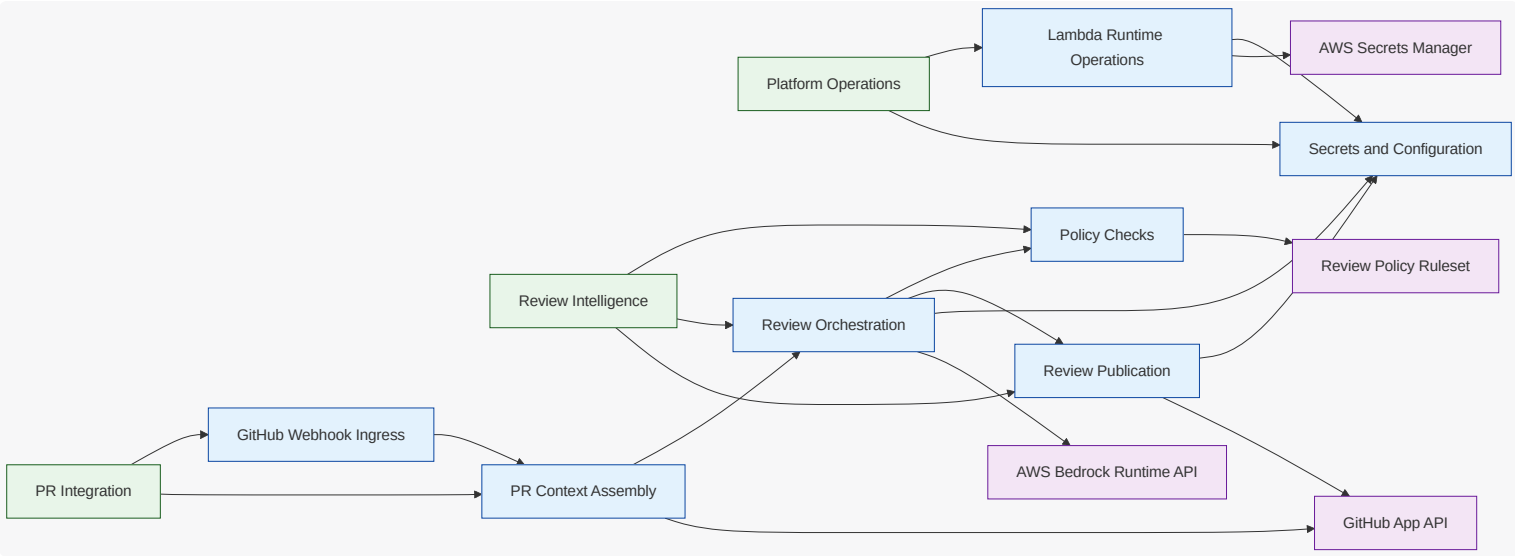
7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and `platform operations`.

7.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are platform ownership boundaries and missing ownership signals?

7.2. View-Scoped Projection



Node ID	Kind	Label
`FG-PLATFORM-OPERATIONS`	functional_group	`Platform Operations`
`FG-PR-INTEGRATION`	functional_group	`PR Integration`
`FG-REVIEW-INTELLIGENCE`	functional_group	`Review Intelligence`
`FU-GITHUB-WEBHOOK-INGRESS`	functional_unit	`GitHub Webhook Ingress`
`FU-LAMBDA-RUNTIME-OPERATIONS`	functional_unit	`Lambda Runtime Operations`
`FU-POLICY-CHECKS`	functional_unit	`Policy Checks`

Node ID	Kind	Label
`FU-PR-CONTEXT-ASSEMBLY`	functional_unit	`PR Context Assembly`
`FU-REVIEW-ORCHESTRATION`	functional_unit	`Review Orchestration`
`FU-REVIEW-PUBLICATION`	functional_unit	`Review Publication`
`FU-SECRETS-CONFIGURATION`	functional_unit	`Secrets and Configuration`
`REF-AWS-BEDROCK-RUNTIME`	referenced_element	AWS `Bedrock Runtime` API
`REF-AWS-SECRETS-MANAGER`	referenced_element	`AWS Secrets Manager`
`REF-GITHUB-APP-API`	referenced_element	`GitHub App API`
`REF-POLICY-RULESET`	referenced_element	Review `Policy Ruleset`

Mapping Type	From	To
contains	`FG-PLATFORM-OPERATIONS`	`FU-LAMBDA-RUNTIME-OPERATIONS`
contains	`FG-PLATFORM-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
contains	`FG-PR-INTEGRATION`	`FU-GITHUB-WEBHOOK-INGRESS`
contains	`FG-PR-INTEGRATION`	`FU-PR-CONTEXT-ASSEMBLY`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-POLICY-CHECKS`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-REVIEW-ORCHESTRATION`
contains	`FG-REVIEW-INTELLIGENCE`	`FU-REVIEW-PUBLICATION`
depends_on	`FU-GITHUB-WEBHOOK-INGRESS`	`FU-PR-CONTEXT-ASSEMBLY`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`REF-AWS-SECRETS-MANAGER`
depends_on	`FU-POLICY-CHECKS`	`REF-POLICY-RULESET`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`FU-REVIEW-ORCHESTRATION`

Mapping Type	From	To
depends_on	FU-PR-CONTEXT-ASSEMBLY	REF-GITHUB-APP-API
depends_on	FU-REVIEW-ORCHESTRATION	FU-POLICY-CHECKS
depends_on	FU-REVIEW-ORCHESTRATION	FU-REVIEW-PUBLICATION
depends_on	FU-REVIEW-ORCHESTRATION	FU-SECRETS-CONFIGURATION
depends_on	FU-REVIEW-ORCHESTRATION	REF-AWS-BEDROCK-RUNTIME
depends_on	FU-REVIEW-PUBLICATION	FU-SECRETS-CONFIGURATION
depends_on	FU-REVIEW-PUBLICATION	REF-GITHUB-APP-API

7.3. Coverage Gaps

Coverage summary: 3/7 units have direct evidence coverage in this view.

- `Lambda Runtime Operations` has no direct inferred runtime/deployment evidence yet.
- `PR Context Assembly` has no direct inferred runtime/deployment evidence yet.
- `Policy Checks` has no direct inferred runtime/deployment evidence yet.
- `Secrets and Configuration` has no direct inferred runtime/deployment evidence yet.

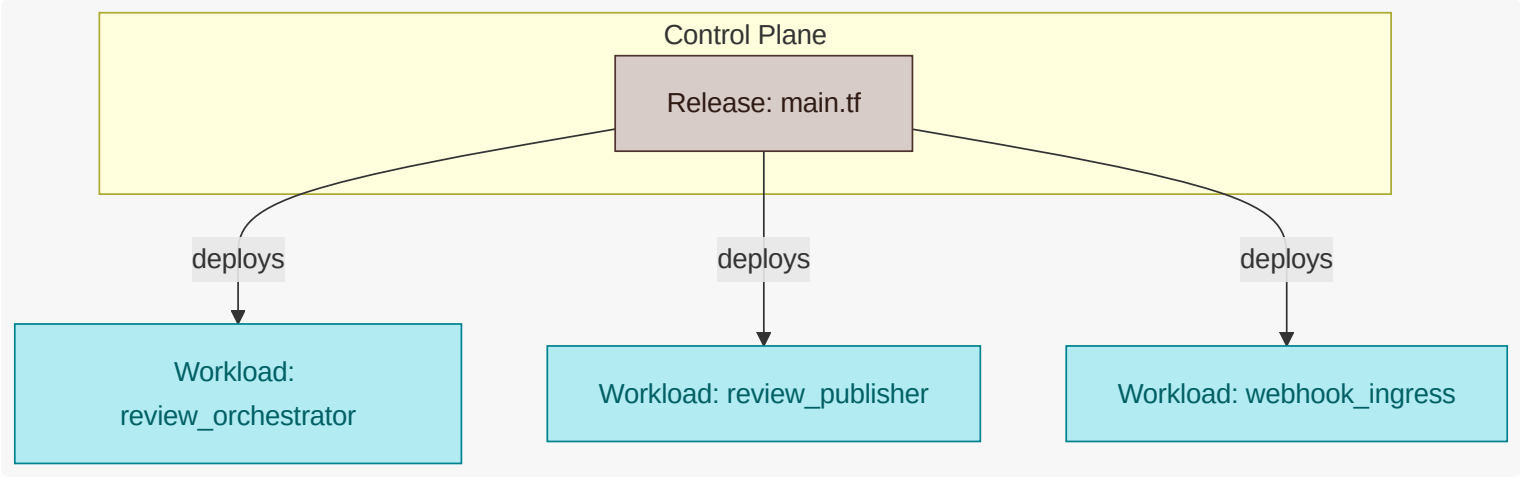
7.4. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/Helm objects for clearer platform ownership mapping.
- Ensure deployment control artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

7.5. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

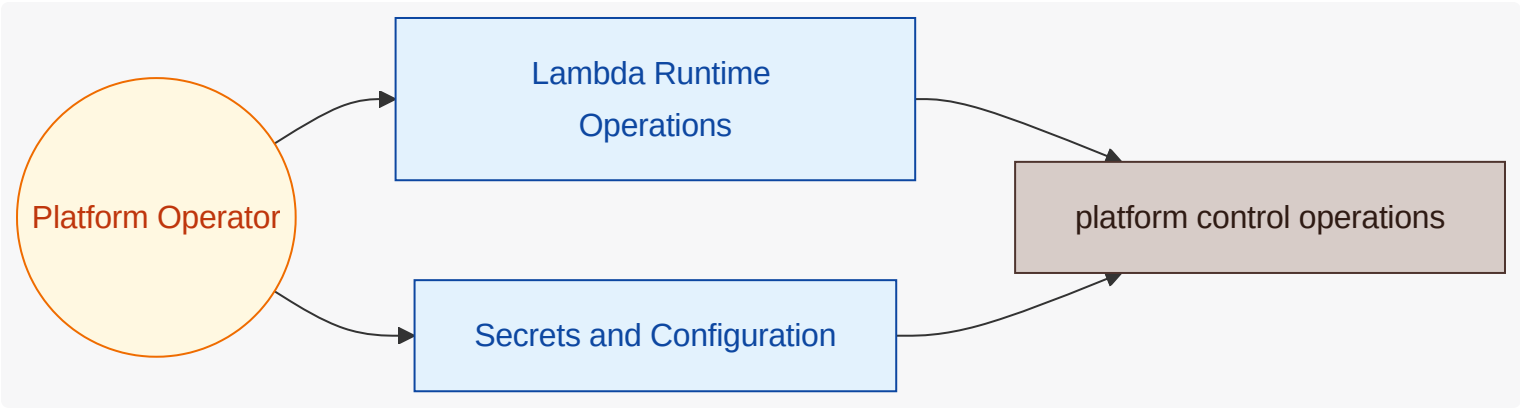
> **NOTE**
> This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. Node labels include role hints (for example `Source`, `Kustomization`, `Release`, `Workload`, `ns/…`) to distinguish similarly named elements.



7.6. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.7. Functional Groups and Units (View Scoped)

7.7.1. PR Integration (FG-PR-INTEGRATION)

`PR Integration` functions deploy independently to isolate `webhook ingress` from context-fetch implementation changes.

GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

GitHub Webhook Ingress Deployment

Deploys as an independently managed Lambda runtime boundary.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-001`, `REQ-PRR-008`, `REQ-PRR-010`
Deployment dependencies	`PR Context Assembly`

Deployment Aspect	View-Scoped Detail
Inferred deployment evidence	runtime: `webhook_ingress`

PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

PR Context Assembly Deployment

Deployed separately to allow context shaping changes without ingress disruption.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-002`
Deployment dependencies	`GitHub App API`, `Review Orchestration`
Inferred deployment evidence	code modules: `pr_context_assembly`

7.7.2. Review Intelligence (FG-REVIEW-INTELLIGENCE)

`Review Intelligence` units roll independently to preserve availability and reduce blast radius during model or rule updates.

Policy Checks (FU-POLICY-CHECKS)

Policy Checks Deployment

Deployed independently to evolve policy logic without model coupling.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-004`, `REQ-PRR-006`
Deployment dependencies	Review `Policy Ruleset`
Inferred deployment evidence	code modules: `policy_checks`

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Review Orchestration Deployment

Isolated deployment enables cautious model/config rollout and fallback control.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-003`, `REQ-PRR-006`
Deployment dependencies	AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration`
Inferred deployment evidence	runtime: `review_orchestrator`

Review Publication (FU-REVIEW-PUBLICATION)

Review Publication Deployment

Release isolation protects `publication` reliability during other unit rollouts.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-005`, `REQ-PRR-006`, `REQ-PRR-007`, `REQ-PRR-008`, `REQ-PRR-009`, `REQ-PRR-010`
Deployment dependencies	`GitHub App API`, `Secrets and Configuration`
Inferred deployment evidence	runtime: `review_publisher`

7.7.3. Platform Operations (FG-PLATFORM-OPERATIONS)

`Platform Operations` governs release `orchestration` and runtime placement to keep review capacity stable.

Lambda Runtime Operations (FU-LAMBDA-RUNTIME-OPERATIONS)

Lambda Runtime Operations Deployment

Governs source-to-release pipeline behavior and runtime placement boundaries.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored `requirement` trigger
Deployment dependencies	`AWS Secrets Manager`, `Secrets and Configuration`
Inferred deployment evidence	code modules: `lambda_runtime_ops`

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Secrets and Configuration Deployment

Managed as a shared operational boundary with controlled rotation and rollout flow.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PRR-007`
Deployment dependencies	none
Inferred deployment evidence	code modules: `secrets_configuration`

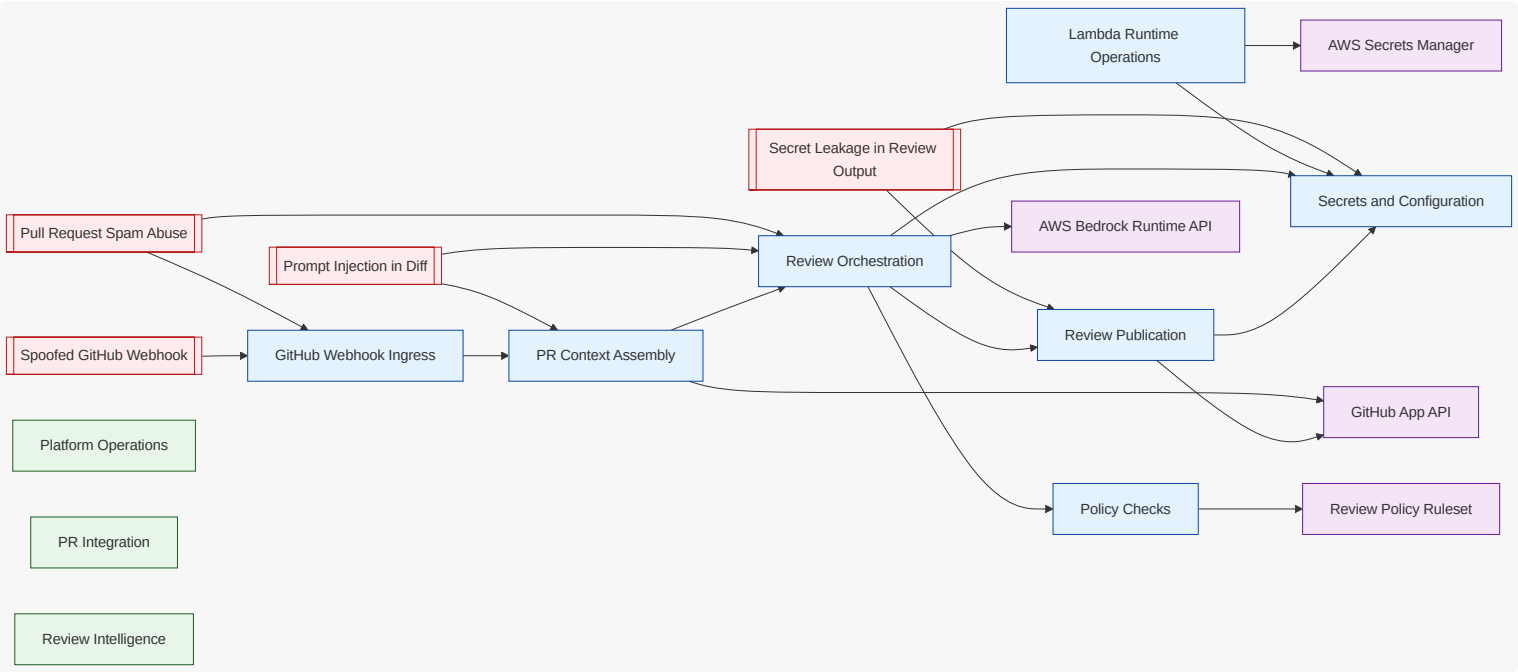
8. Security View

Show inferred exposure and dependency risk points aligned to unit boundaries, focused on attack paths and security evidence.

8.1. What This View Answers

- What are the main trust boundaries and attack paths?
- Which units are exposed and what security evidence exists?
- Where is the threat model incomplete?

8.2. View-Scoped Projection



Node ID	Kind	Label
AV-PR-SPAM-ABUSE	attack_vector	Pull Request Spam Abuse
AV-PROMPT-INJECTION-IN-DIFF	attack_vector	Prompt Injection in Diff
AV-SECRET-LEAKAGE-IN-REVIEW	attack_vector	Secret Leakage in Review Output
AV-SPOOFED-GITHUB-WEBHOOK	attack_vector	Spoofed GitHub Webhook
FG-PLATFORM-OPERATIONS	functional_group	Platform Operations
FG-PR-INTEGRATION	functional_group	PR Integration

Node ID	Kind	Label
`FG-REVIEW-INTELLIGENCE`	functional_group	`Review Intelligence`
`FU-GITHUB-WEBHOOK-INGRESS`	functional_unit	`GitHub Webhook Ingress`
`FU-LAMBDA-RUNTIME-OPERATIONS`	functional_unit	`Lambda Runtime Operations`
`FU-POLICY-CHECKS`	functional_unit	`Policy Checks`
`FU-PR-CONTEXT-ASSEMBLY`	functional_unit	`PR Context Assembly`
`FU-REVIEW-ORCHESTRATION`	functional_unit	`Review Orchestration`
`FU-REVIEW-PUBLICATION`	functional_unit	`Review Publication`
`FU-SECRETS-CONFIGURATION`	functional_unit	`Secrets and Configuration`
`REF-AWS-BEDROCK-RUNTIME`	referenced_element	AWS `Bedrock Runtime` API
`REF-AWS-SECRETS-MANAGER`	referenced_element	`AWS Secrets Manager`
`REF-GITHUB-APP-API`	referenced_element	`GitHub App API`
`REF-POLICY-RULESET`	referenced_element	Review `Policy Ruleset`

Mapping Type	From	To
depends_on	`FU-GITHUB-WEBHOOK-INGRESS`	`FU-PR-CONTEXT-ASSEMBLY`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`REF-AWS-SECRETS-MANAGER`
depends_on	`FU-POLICY-CHECKS`	`REF-POLICY-RULESET`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`FU-REVIEW-ORCHESTRATION`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`REF-GITHUB-APP-API`

Mapping Type	From	To
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-POLICY-CHECKS`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-REVIEW-PUBLICATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`REF-AWS-BEDROCK-RUNTIME`
depends_on	`FU-REVIEW-PUBLICATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-PUBLICATION`	`REF-GITHUB-APP-API`
targets	`AV-PR-SPAM-ABUSE`	`FU-GITHUB-WEBHOOK-INGRESS`
targets	`AV-PR-SPAM-ABUSE`	`FU-REVIEW-ORCHESTRATION`
targets	`AV-PROMPT-INJECTION-IN-DIFF`	`FU-PR-CONTEXT-ASSEMBLY`
targets	`AV-PROMPT-INJECTION-IN-DIFF`	`FU-REVIEW-ORCHESTRATION`
targets	`AV-SECRET-LEAKAGE-IN-REVIEW`	`FU-REVIEW-PUBLICATION`
targets	`AV-SECRET-LEAKAGE-IN-REVIEW`	`FU-SECRETS-CONFIGURATION`
targets	`AV-SPOOFED-GITHUB-WEBHOOK`	`FU-GITHUB-WEBHOOK-INGRESS`

8.3. Coverage Gaps

Coverage summary: 5/7 units have direct evidence coverage in this view.

- `Lambda Runtime Operations` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.
- `Policy Checks` has no explicit authored attack-vector mapping yet; document technical and fraud/abuse threats even when mitigated.

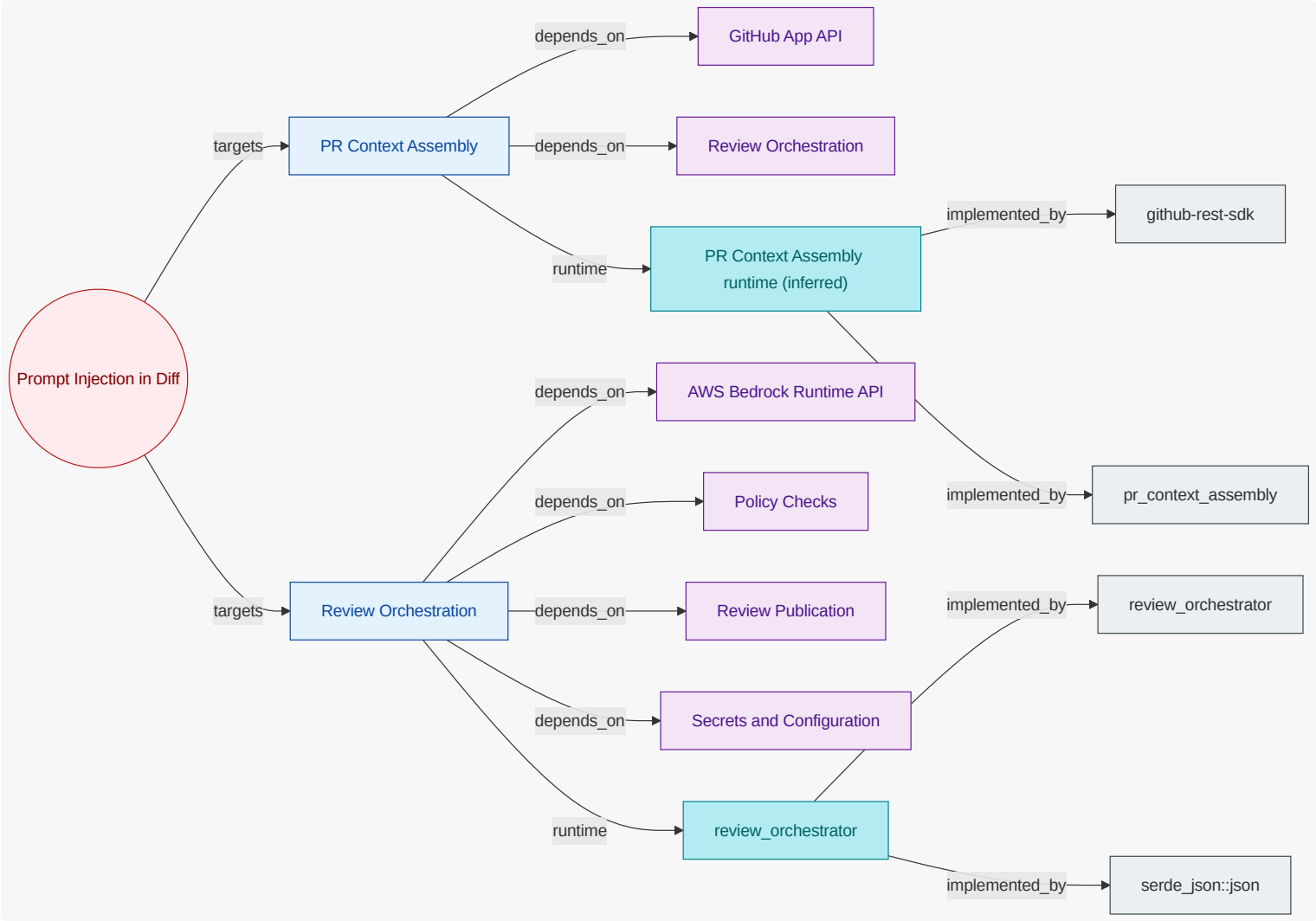
8.4. Recommended Next Evidence Additions

- Add explicit authored attack-vector mappings for units currently marked without threat linkage.
- Add security signal ownership metadata (logs/alerts/events) for unresolved observability evidence.
- Map additional security-relevant dependencies and exposures into `authored mappings` for traceable coverage.

8.5. Attack Vector Chapters

8.5.1. `Prompt Injection in Diff` (AV-PROMPT-INJECTION-IN-DIFF)

Malicious prompt-like content embedded in pull request diffs.



PR Context Assembly (FU-PR-CONTEXT-ASSEMBLY)

Applies contextual sanitization to reduce `prompt injection` risk from diffs.

Attack Surface

`GitHub App API`, `Review Orchestration`

Security Evidence Present

code modules: `pr_context_assembly`

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Enforces safe prompt construction and bounded output handling under adversarial input.

Attack Surface

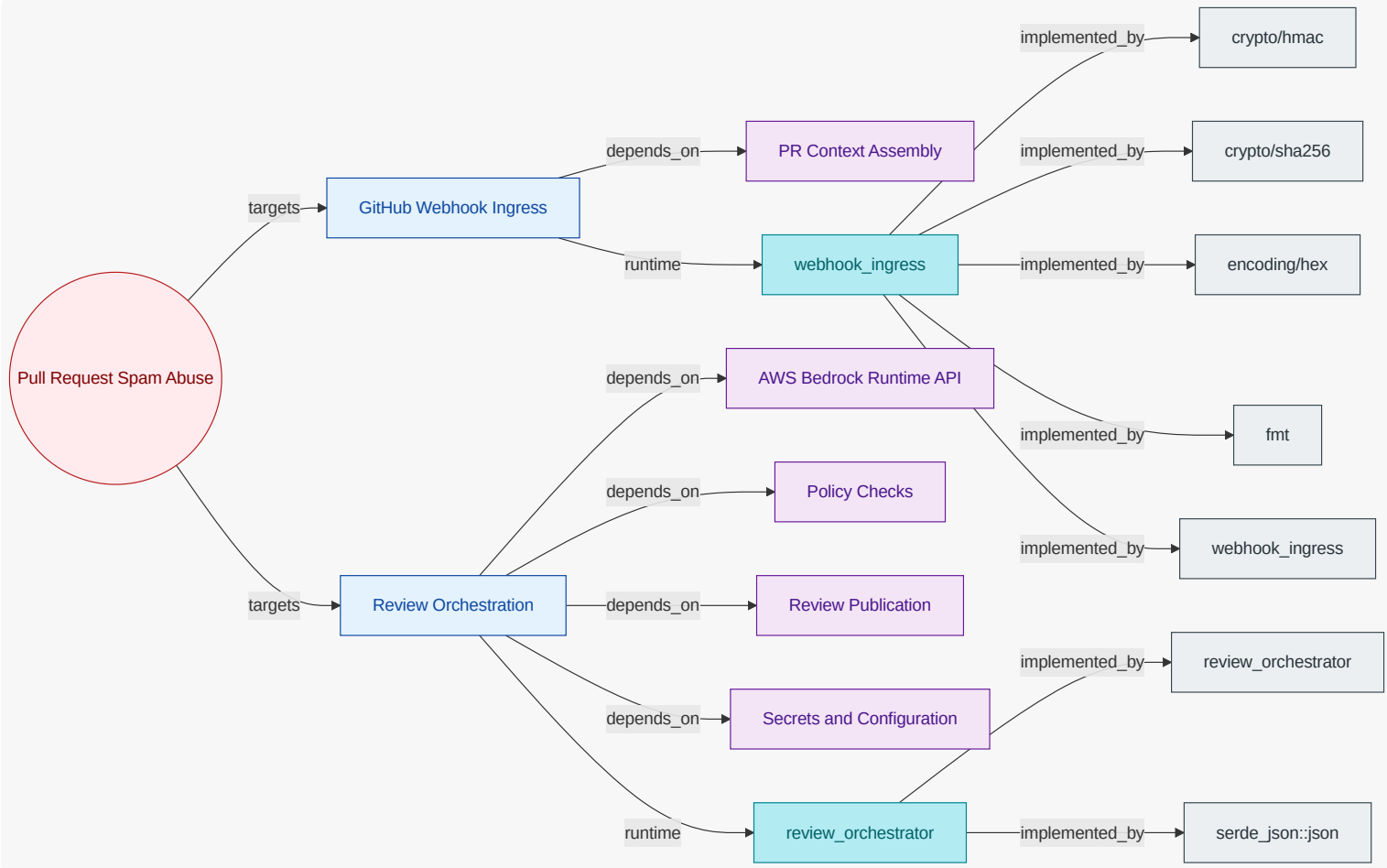
AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration`

Security Evidence Present

runtime: `review_orchestrator` | code modules: `review_orchestrator`

8.5.2. `Pull Request Spam Abuse` (AV-PR-SPAM-ABUSE)

High-rate PR event flooding that can exhaust review capacity.



GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Enforces signature verification and rejects spoofed or malformed events.

Attack Surface

`PR Context Assembly`

Security Evidence Present

runtime: `webhook_ingress` | code modules: `webhook_ingress`

Review Orchestration (FU-REVIEW-ORCHESTRATION)

Enforces safe prompt construction and bounded output handling under adversarial input.

Attack Surface

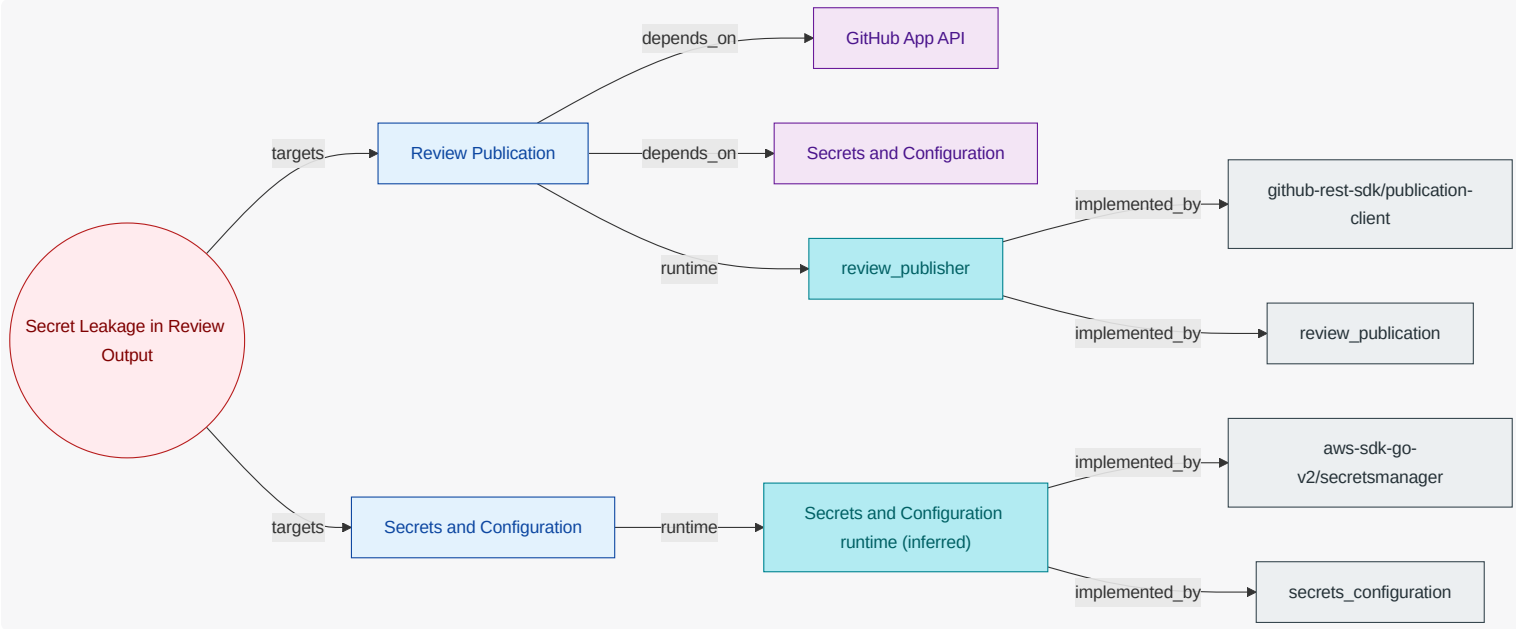
AWS `Bedrock Runtime` API, `Policy Checks`, `Review Publication`, `Secrets and Configuration`

Security Evidence Present

runtime: `review_orchestrator` | code modules: `review_orchestrator`

8.5.3. Secret Leakage in Review Output (AV-SECRET-LEAKAGE-IN-REVIEW)

Generated findings accidentally exposing secrets/tokens in comments.



Review Publication (FU-REVIEW-PUBLICATION)

Applies redaction and policy-aware output filtering before publishing findings.

Attack Surface

`GitHub App API`, `Secrets and Configuration`

Security Evidence Present

runtime: `review_publisher` | code modules: `review_publication`

Secrets and Configuration (FU-SECRETS-CONFIGURATION)

Enforces credential isolation, least-privilege access, and redaction-safe metadata handling.

Attack Surface

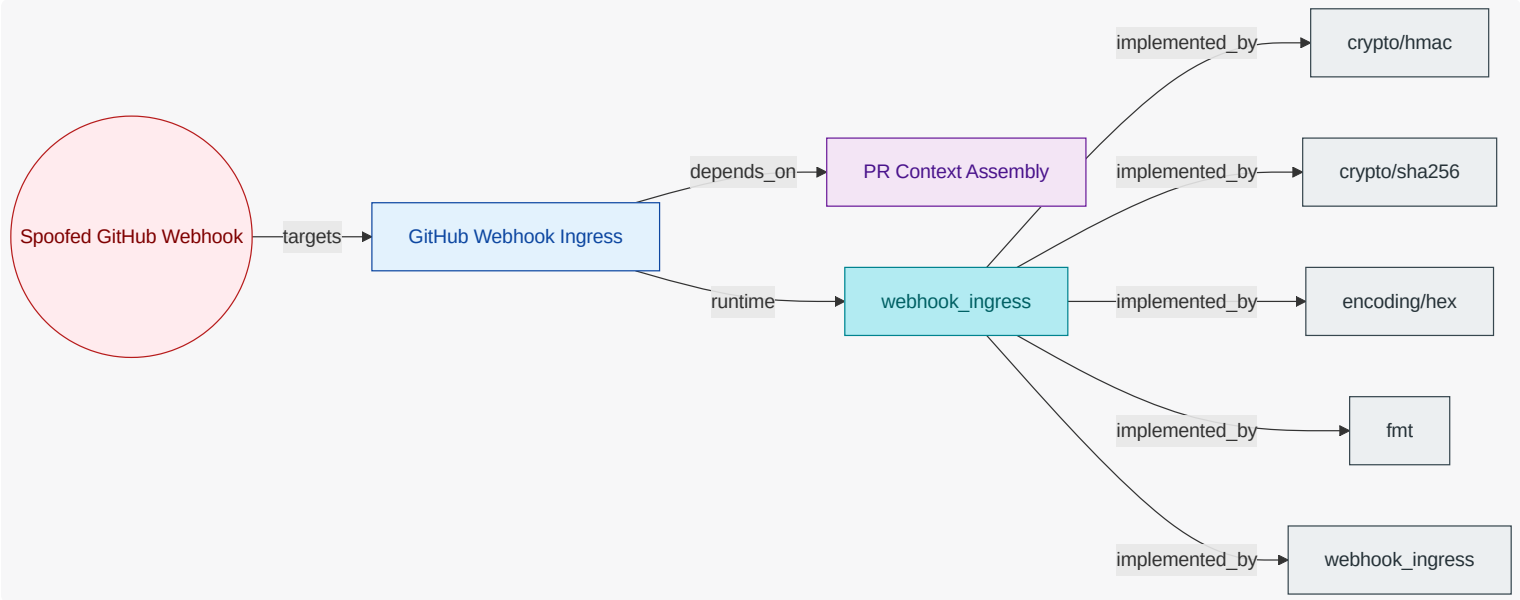
none

Security Evidence Present

code modules: `secrets_configuration`

8.5.4. `Spoofed GitHub Webhook` (AV-SPOOFED-GITHUB-WEBHOOK)

Forged webhook payload attempting unauthorized review execution.



GitHub Webhook Ingress (FU-GITHUB-WEBHOOK-INGRESS)

Enforces signature verification and rejects spoofed or malformed events.

Attack Surface

`PR Context Assembly`

Security Evidence Present

runtime: `webhook_ingress` | code modules: `webhook_ingress`

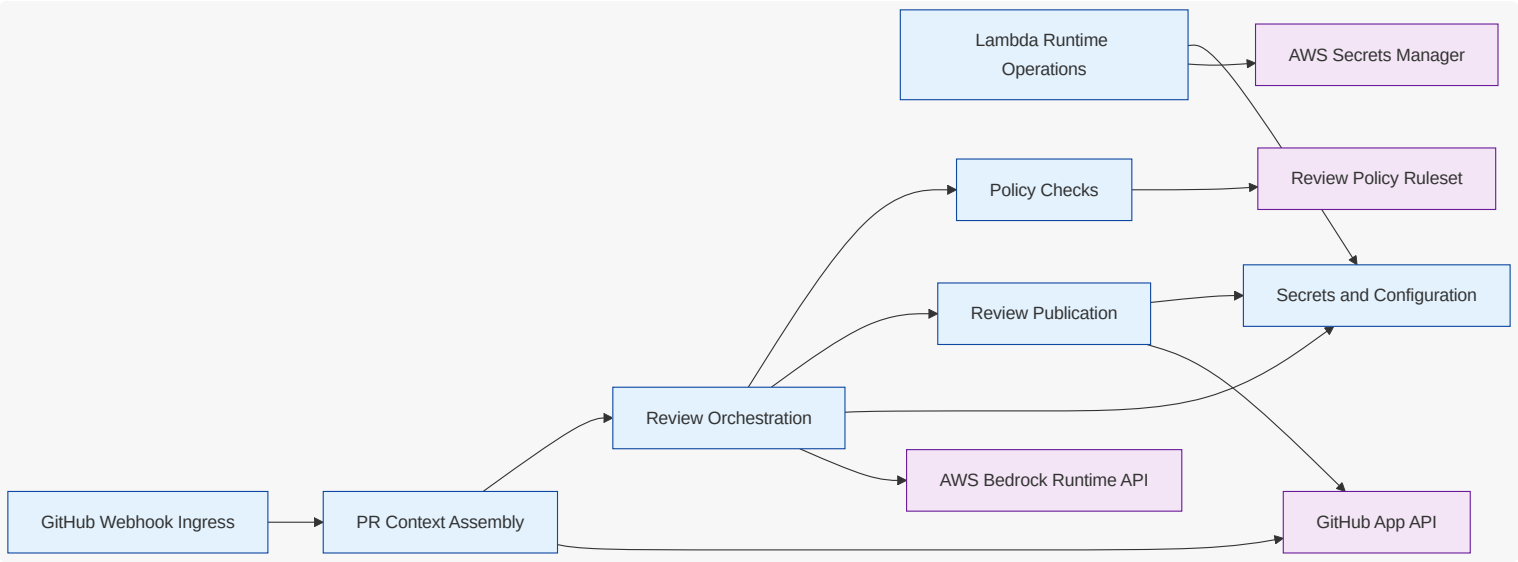
9. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit evidence gaps.

9.1. What This View Answers

- Which `requirements` map to which units?
- Which authored units have runtime/code evidence and which do not?
- Where are confidence or completeness gaps in trace coverage?

9.2. View-Scoped Projection



Node ID	Kind	Label
`FU-GITHUB-WEBHOOK-INGRESS`	functional_unit	`GitHub Webhook Ingress`
`FU-LAMBDA-RUNTIME-OPERATIONS`	functional_unit	`Lambda Runtime Operations`
`FU-POLICY-CHECKS`	functional_unit	`Policy Checks`
`FU-PR-CONTEXT-ASSEMBLY`	functional_unit	`PR Context Assembly`
`FU-REVIEW-ORCHESTRATION`	functional_unit	`Review Orchestration`
`FU-REVIEW-PUBLICATION`	functional_unit	`Review Publication`

Node ID	Kind	Label
`FU-SECRETS-CONFIGURATION`	functional_unit	`Secrets and Configuration`
`REF-AWS-BEDROCK-RUNTIME`	referenced_element	AWS `Bedrock Runtime` API
`REF-AWS-SECRETS-MANAGER`	referenced_element	`AWS Secrets Manager`
`REF-GITHUB-APP-API`	referenced_element	`GitHub App API`
`REF-POLICY-RULESET`	referenced_element	Review `Policy Ruleset`

Mapping Type	From	To
depends_on	`FU-GITHUB-WEBHOOK-INGRESS`	`FU-PR-CONTEXT-ASSEMBLY`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-LAMBDA-RUNTIME-OPERATIONS`	`REF-AWS-SECRETS-MANAGER`
depends_on	`FU-POLICY-CHECKS`	`REF-POLICY-RULESET`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`FU-REVIEW-ORCHESTRATION`
depends_on	`FU-PR-CONTEXT-ASSEMBLY`	`REF-GITHUB-APP-API`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-POLICY-CHECKS`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-REVIEW-PUBLICATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-ORCHESTRATION`	`REF-AWS-BEDROCK-RUNTIME`
depends_on	`FU-REVIEW-PUBLICATION`	`FU-SECRETS-CONFIGURATION`
depends_on	`FU-REVIEW-PUBLICATION`	`REF-GITHUB-APP-API`

9.3. Coverage Gaps

Coverage summary: 7/7 units have direct evidence coverage in this view.

- `Lambda Runtime Operations` has no direct inferred runtime evidence yet.
- `PR Context Assembly` has no direct inferred runtime evidence yet.
- `Policy Checks` has no direct inferred runtime evidence yet.

- `Secrets and Configuration` has no direct inferred runtime evidence yet.

9.4. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where `requirement` scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code evidence.

9.5. Requirement-to-Unit Mapping (Compact)

What this matrix shows: authored requirement-to-unit mappings in a compact square layout (requirements as rows, functional units as columns).

Requirement	FU-GITHUB-WEBHOOK-INGRESS	FU-POLICY-CHECKS	FU-PR-CONTEXT-ASSEMBLY	FU-REVIEW-ORCHESTRATION	FU-REVIEW-PUBLICATION	FU-SECRETS-CONFIGURATION
REQ-PRR-001	X					
REQ-PRR-002			X			
REQ-PRR-003				X		
REQ-PRR-004		X				
REQ-PRR-005					X	
REQ-PRR-006		X		X	X	
REQ-PRR-007					X	X
REQ-PRR-008	X				X	
REQ-PRR-009					X	
REQ-PRR-010	X				X	

9.6. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

Check ID	Requirement	Result	Evidence / Notes
`VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5`	`REQ-PRR-001`	pass	test-results/e2e-pr-opened-review.xml
`VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5`	`REQ-PRR-002`	pass	test-results/e2e-pr-opened-review.xml

Check ID	Requirement	Result	Evidence / Notes
`VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5`	`REQ-PRR-003`	pass	test-results/e2e-pr-opened-review.xml
`VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A`	`REQ-PRR-004`	pass	test-results/integration-policy-fallback.json deterministic `policy checks` executed
`VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5`	`REQ-PRR-005`	pass	test-results/contract-checkrun.json check run payload includes summary and neutral conclusion
`VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6`	`REQ-PRR-005`	not-run	none No parsed test-result artifact for this `requirement` yet.
`VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A`	`REQ-PRR-006`	pass	test-results/integration-policy-fallback.json policy-only fallback published
`VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1`	`REQ-PRR-007`	pass	test-results/audit-redaction-report.md
`VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2`	`REQ-PRR-008`	pass	test-results/unit-rate-limit.xml

10. Traceability Appendix

10.1. Requirement Details

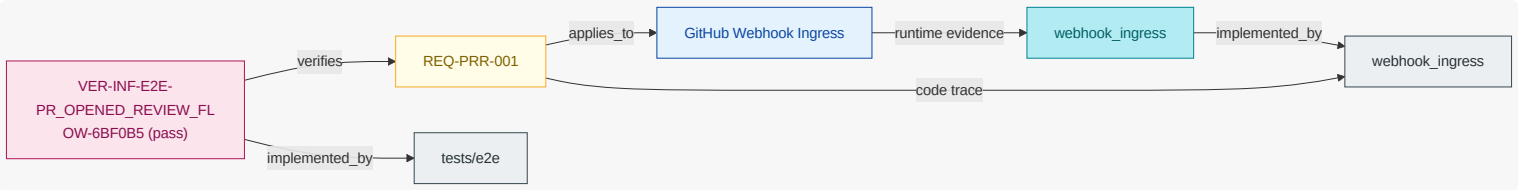
10.1.1. REQ-PRR-001

While `pull request review is active`, when `pull request synchronize event is received`, `the pr review system` shall verify `webhook signature` before processing review workflow.

> **NOTE** Webhook trust boundary validation.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



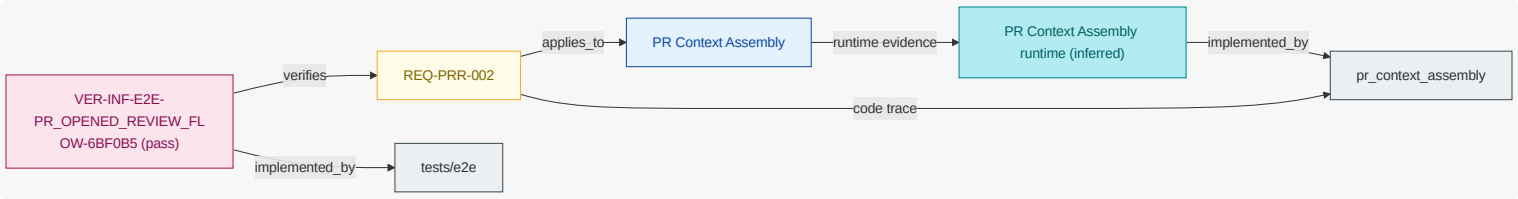
10.1.2. REQ-PRR-002

While `pull request review is active`, when `pull request opened event is received`, `the pr review system` shall construct `diff context` from changed files for review execution.

> **NOTE** Context ingestion from `GitHub API`.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



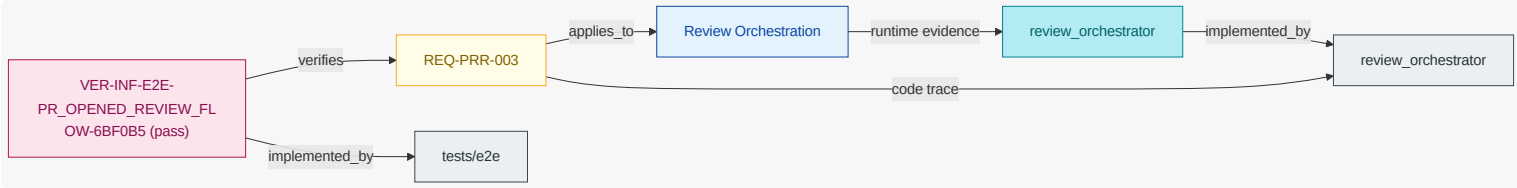
10.1.3. REQ-PRR-003

Where `ai review is enabled`, when `diff context is prepared`, `the pr review system` shall request `bedrock runtime` analysis for review findings.

> **NOTE** AI-assisted review path.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



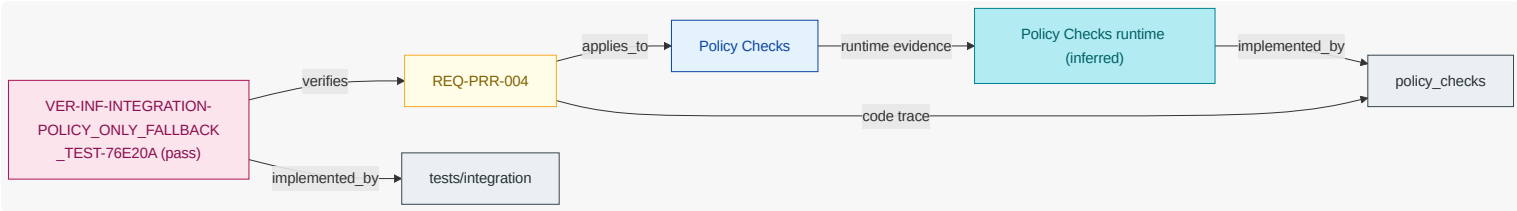
10.1.4. REQ-PRR-004

Where `policy checks are enabled`, when `pull request review is active`, `the pr review system` shall evaluate deterministic `policy checks` for secret exposure plus infrastructure risk plus dependency changes.

> **NOTE** Deterministic non-LLM review checks.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



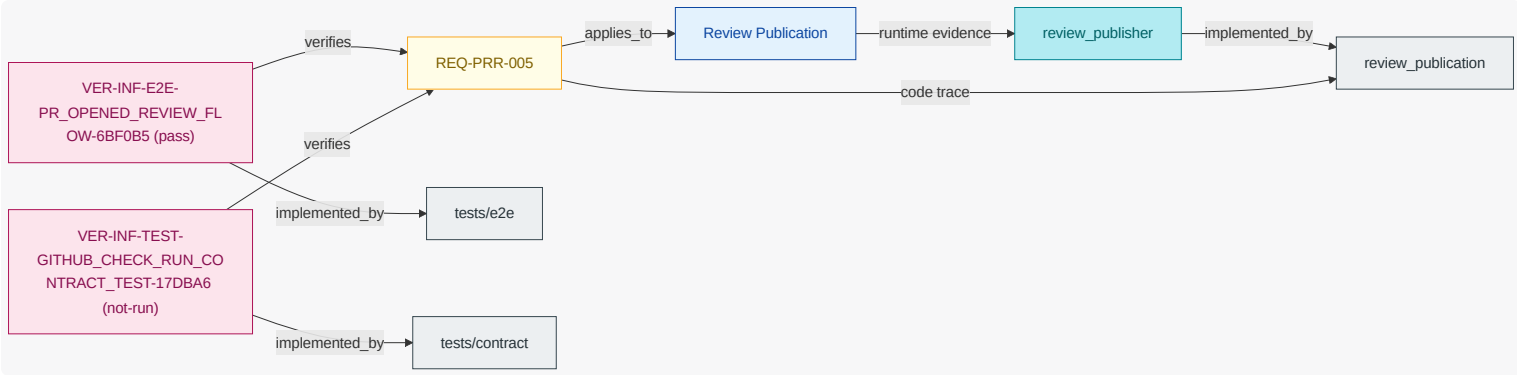
10.1.5. REQ-PRR-005

While `pull request review is active`, when `review findings are generated`, `the pr review system` shall publish `check run summary` to `repository maintainer`.

> **NOTE** Result `publication` contract.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



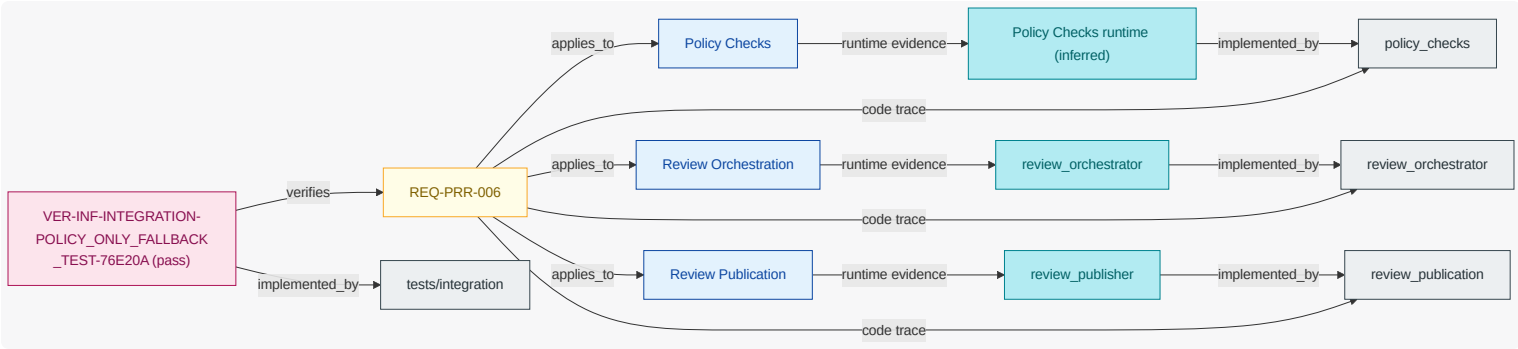
10.1.6. REQ-PRR-006

If `bedrock runtime is unavailable`, then `the pr review system` shall publish deterministic policy-only `review outcome`.

> **NOTE** Graceful degradation `requirement`.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



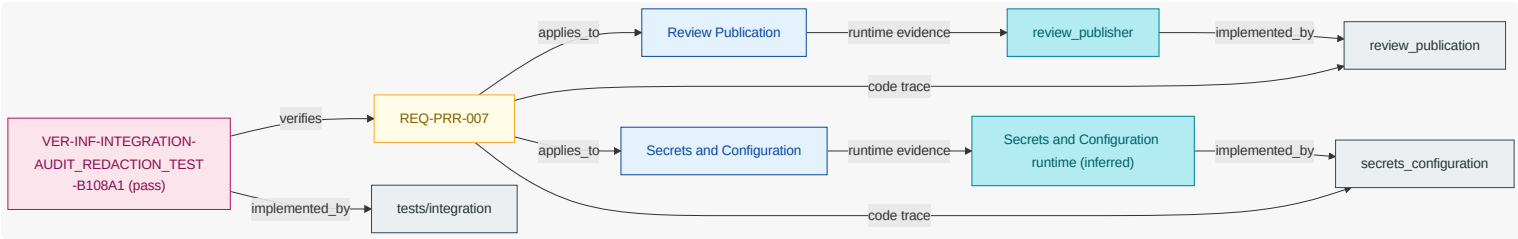
10.1.7. REQ-PRR-007

Where `audit logging is enabled`, when `review findings are published`, `the pr review system` shall persist `redacted audit metadata` for audit traceability.

> **NOTE** Evidence and audit lineage.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



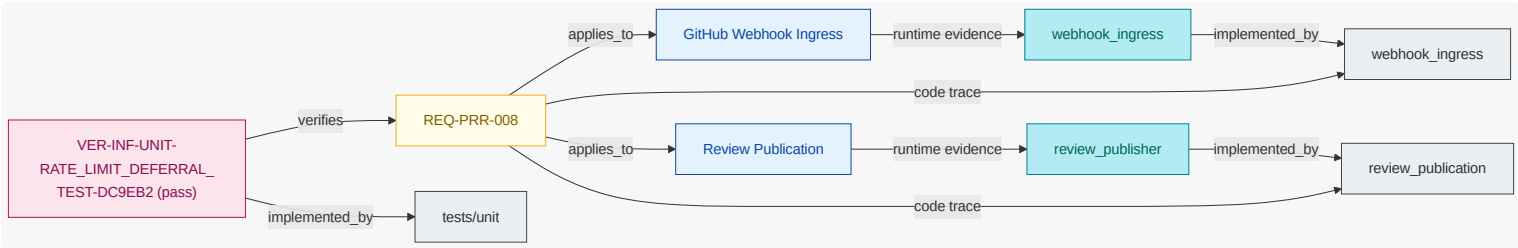
10.1.8. REQ-PRR-008

If `review request rate limit is exceeded`, then `the pr review system` shall defer processing.

> **NOTE** Abuse and capacity protection behavior.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



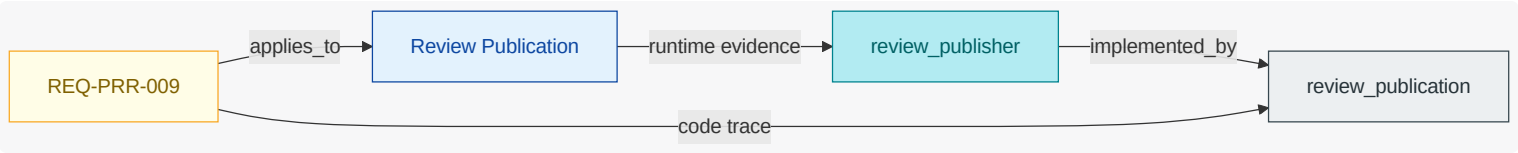
10.1.9. REQ-PRR-009

While `pull request review is active`, when `review findings are generated`, `the pr review system` shall publish inline comments to `repository maintainer`.

> **NOTE** Inline-annotation `publication` contract.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



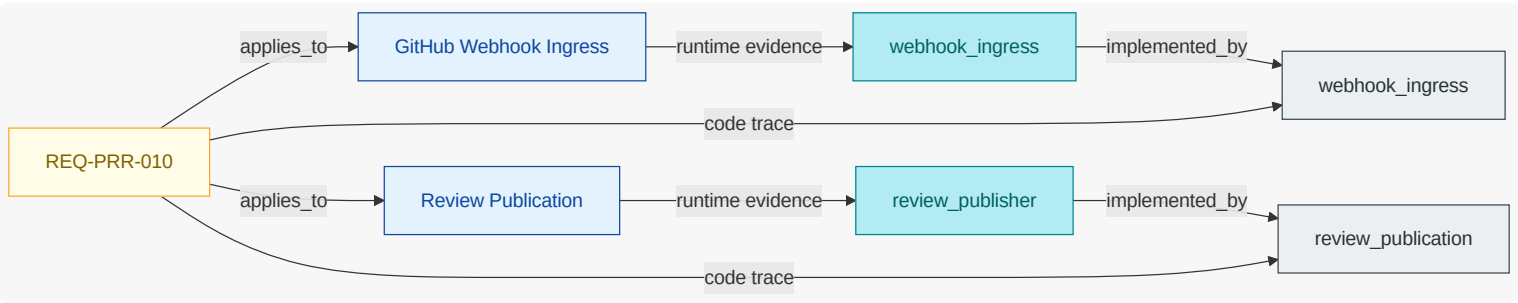
10.1.10. REQ-PRR-010

If `review request rate limit is exceeded`, then `the pr review system` shall notify `repository maintainer`.

> **NOTE** `Maintainer` notification for deferred review processing.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



10.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

10.2.1. Pr Opened Review Flow

Field	Value
ID	`VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5`
Kind	e2e
Status	pass
Verifies Requirements	`REQ-PRR-001`, `REQ-PRR-002`, `REQ-PRR-003`, `REQ-PRR-005`

Field	Value
Test Code Elements	tests/e2e/pr_opened_review_flow.yaml
Derived Functional Owners	`FU-GITHUB-WEBHOOK-INGRESS`, `FU-PR-CONTEXT-ASSEMBLY`, `FU-REVIEW-ORCHESTRATION`, `FU-REVIEW-PUBLICATION`
Result Summary	pass:4
Evidence	test-results/contract-checkrun.json, test-results/e2e-pr-opened-review.xml, tests/e2e/pr_opened_review_flow.yaml
Notes	exercises end-to-end pull request opened flow from webhook ingest to published review

10.2.2. Audit Redaction Test

Field	Value
ID	`VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1`
Kind	integration
Status	pass
Verifies Requirements	`REQ-PRR-007`
Test Code Elements	tests/integration/audit_redaction_test.ts
Derived Functional Owners	`FU-REVIEW-PUBLICATION`, `FU-SECRETS-CONFIGURATION`
Result Summary	pass:1
Evidence	test-results/audit-redaction-report.md, tests/integration/audit_redaction_test.ts
Notes	checks audit evidence redaction for secrets and sensitive payload fields

10.2.3. Policy Only Fallback Test

Field	Value
ID	`VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A`
Kind	integration
Status	pass
Verifies Requirements	`REQ-PRR-004`, `REQ-PRR-006`
Test Code Elements	tests/integration/policy_only_fallback_test.rs

Field	Value
Derived Functional Owners	`FU-POLICY-CHECKS`, `FU-REVIEW-ORCHESTRATION`, `FU-REVIEW-PUBLICATION`
Result Summary	pass:2
Evidence	test-results/integration-policy-fallback.json, tests/integration/policy_only_fallback_test.rs
Notes	verifies policy-only fallback when Bedrock inference is unavailable

10.2.4. Github Check Run Contract Test

Field	Value
ID	`VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6`
Kind	test
Status	not-run
Verifies Requirements	`REQ-PRR-005`
Test Code Elements	tests/contract/github_check_run_contract_test.go
Derived Functional Owners	`FU-REVIEW-PUBLICATION`
Result Summary	not-run:1
Evidence	tests/contract/github_check_run_contract_test.go
Notes	validates GitHub check-run contract shape and required review result fields

10.2.5. Rate Limit Deferral Test

Field	Value
ID	`VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2`
Kind	unit
Status	pass
Verifies Requirements	`REQ-PRR-008`
Test Code Elements	tests/unit/rate_limit_deferral_test.go
Derived Functional Owners	`FU-GITHUB-WEBHOOK-INGRESS`, `FU-REVIEW-PUBLICATION`

Field	Value
Result Summary	pass:1
Evidence	test-results/unit-rate-limit.xml, tests/unit/rate_limit_deferral_test.go
Notes	verifies rate-limit deferral behavior for pull request review processing

11. Reference Index

11.1. Authored References

REQ-PRR-001

While pull request review is active, when pull request synchronize event is received, the pr review system shall verify webhook signature before processing review workflow.

Key	Value
Kind	Requirement
Name	REQ-PRR-001
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Pr Opened Review Flow`

REQ-PRR-002

While pull request review is active, when pull request opened event is received, the pr review system shall construct diff context from changed files for review execution.

Key	Value
Kind	Requirement
Name	REQ-PRR-002
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Pr Opened Review Flow`

REQ-PRR-003

Where ai review is enabled, when diff context is prepared, the pr review system shall request bedrock runtime analysis for review findings.

Key	Value
Kind	Requirement
Name	REQ-PRR-003
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Pr Opened Review Flow`

REQ-PRR-004

Where policy checks are enabled, when pull request review is active, the pr review system shall evaluate deterministic policy checks for secret exposure plus infrastructure risk plus dependency changes.

Key	Value
Kind	Requirement
Name	REQ-PRR-004
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Policy Only Fallback Test`

REQ-PRR-005

While pull request review is active, when review findings are generated, the pr review system shall publish check run summary to repository maintainer.

Key	Value
Kind	Requirement
Name	REQ-PRR-005
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Pr Opened Review Flow`, `Github Check Run Contract Test`

REQ-PRR-006

If bedrock runtime is unavailable, then the pr review system shall publish deterministic policy-only review outcome.

Key	Value
Kind	Requirement
Name	REQ-PRR-006
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Policy Only Fallback Test`

REQ-PRR-007

Where audit logging is enabled, when review findings are published, the pr review system shall persist redacted audit metadata for audit traceability.

Key	Value
Kind	Requirement
Name	REQ-PRR-007
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Audit Redaction Test`

REQ-PRR-008

If review request rate limit is exceeded, then the pr review system shall defer processing.

Key	Value
Kind	Requirement
Name	REQ-PRR-008
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Rate Limit Deferral Test`

REQ-PRR-009

While pull request review is active, when review findings are generated, the pr review system shall publish inline comments to repository maintainer.

Key	Value
Kind	Requirement
Name	REQ-PRR-009
Mentioned In	`Architecture Intent View`, `Deployment View`

REQ-PRR-010

If review request rate limit is exceeded, then the pr review system shall notify repository maintainer.

Key	Value
Kind	Requirement
Name	REQ-PRR-010
Mentioned In	`Architecture Intent View`, `Deployment View`

11.2. Catalog References

ACT-DEVELOPER

Engineer opening and updating pull requests.

Key	Value
Kind	Catalog Actor
Name	developer
Mentioned In	`Architecture Intent View`, `Communication View`

ACT-PLATFORM-OPERATOR

Owner of deployment/runtime operations and reliability controls.

Key	Value
Kind	Catalog Actor
Name	platform operator
Mentioned In	`Architecture Intent View`, `Communication View`

ACT-REPOSITORY-MAINTAINER

Owner of merge decisions and repository quality gates.

Key	Value
Kind	Catalog Actor
Name	repository maintainer
Mentioned In	`Architecture Intent View`, `Communication View`, `REQ-PRR-005`, `REQ-PRR-009`, `REQ-PRR-010`

ACT-SECURITY-ENGINEER

Owner of security policy checks and finding triage.

Key	Value
Kind	Catalog Actor
Name	security engineer
Mentioned In	`Architecture Intent View`, `Communication View`

ai review enabled

Feature flag enabling model-assisted review.

Key	Value
Kind	Catalog Alias
Name	ai review enabled
Mentioned In	n/a

audit enabled

Feature flag enabling audit metadata persistence.

Key	Value
Kind	Catalog Alias

Key	Value
Name	audit enabled
Mentioned In	n/a

authorized events

Event raised when new commits are pushed to an open pull request.

Key	Value
Kind	Catalog Alias
Name	authorized events
Mentioned In	n/a

aws sdk

SDK dependency used for AWS runtime integrations.

Key	Value
Kind	Catalog Alias
Name	aws sdk
Mentioned In	n/a

bedrock api

External Bedrock runtime endpoint used for model inference.

Key	Value
Kind	Catalog Alias
Name	bedrock api
Mentioned In	n/a

bedrock unavailable

Event raised when Bedrock inference path is degraded.

Key	Value
Kind	Catalog Alias
Name	bedrock unavailable

Key	Value
Mentioned In	n/a

check run summaries

Aggregated review result posted as a GitHub check run.

Key	Value
Kind	Catalog Alias
Name	check run summaries
Mentioned In	n/a

context assembly

Fetches changed files and prepares bounded review context payloads.

Key	Value
Kind	Catalog Alias
Name	context assembly
Mentioned In	`Architecture Intent View`

context prepared

Event indicating context assembly completed and is ready for analysis.

Key	Value
Kind	Catalog Alias
Name	context prepared
Mentioned In	n/a

deterministic checks

Evaluates deterministic repository policy checks.

Key	Value
Kind	Catalog Alias
Name	deterministic checks
Mentioned In	`Architecture Intent View`

findings generated

Event indicating analysis output is ready for publication.

Key	Value
Kind	Catalog Alias
Name	findings generated
Mentioned In	n/a

findings published

Event indicating review output has been published to GitHub checks/comments.

Key	Value
Kind	Catalog Alias
Name	findings published
Mentioned In	n/a

github api

External GitHub API used for pull request metadata and review publication.

Key	Value
Kind	Catalog Alias
Name	github api
Mentioned In	`REQ-PRR-002`

github integration

Functional group for webhook ingress and pull request context intake.

Key	Value
Kind	Catalog Alias
Name	github integration
Mentioned In	n/a

github sdk

SDK dependency used for GitHub App REST interactions.

Key	Value
Kind	Catalog Alias
Name	github sdk
Mentioned In	n/a

lambda ops

Manages runtime deployment and operational controls.

Key	Value
Kind	Catalog Alias
Name	lambda ops
Mentioned In	n/a

maintainer

Owner of merge decisions and repository quality gates.

Key	Value
Kind	Catalog Alias
Name	maintainer
Mentioned In	n/a

normal mode

Default operating mode using AI plus deterministic checks.

Key	Value
Kind	Catalog Alias
Name	normal mode
Mentioned In	n/a

normalized review context

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Alias

Key	Value
Name	normalized review context
Mentioned In	n/a

orchestration

Coordinates Bedrock analysis with deterministic checks and fallback logic.

Key	Value
Kind	Catalog Alias
Name	orchestration
Mentioned In	`Architecture Intent View`, `Deployment View`

platform ops

Owner of deployment/runtime operations and reliability controls.

Key	Value
Kind	Catalog Alias
Name	platform ops
Mentioned In	n/a

policy bundle

Deterministic review policy bundle used during analysis.

Key	Value
Kind	Catalog Alias
Name	policy bundle
Mentioned In	n/a

policy checks enabled

Feature flag enabling deterministic policy checks.

Key	Value
Kind	Catalog Alias
Name	policy checks enabled

Key	Value
Mentioned In	n/a

pr author

Engineer opening and updating pull requests.

Key	Value
Kind	Catalog Alias
Name	pr author
Mentioned In	n/a

pr opened

Event raised when a new pull request is opened.

Key	Value
Kind	Catalog Alias
Name	pr opened
Mentioned In	n/a

pr spam

High-volume event abuse causing review capacity exhaustion.

Key	Value
Kind	Catalog Alias
Name	pr spam
Mentioned In	n/a

pr synchronize

Event raised when new commits are pushed to an open pull request.

Key	Value
Kind	Catalog Alias
Name	pr synchronize
Mentioned In	n/a

prompt injection

Malicious prompt-like content embedded in pull request code changes.

Key	Value
Kind	Catalog Alias
Name	prompt injection
Mentioned In	`Security View`

publication

Publishes review findings to GitHub checks and comments.

Key	Value
Kind	Catalog Alias
Name	publication
Mentioned In	`Architecture Intent View`, `Deployment View`, `REQ-PRR-005`, `REQ-PRR-009`

publication output

Event indicating review output has been published to GitHub checks/comments.

Key	Value
Kind	Catalog Alias
Name	publication output
Mentioned In	n/a

pull request files

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Alias
Name	pull request files
Mentioned In	n/a

pull request metadata

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Alias
Name	pull request metadata
Mentioned In	n/a

rate limit exceeded

Event raised when PR review rate guardrails are hit.

Key	Value
Kind	Catalog Alias
Name	rate limit exceeded
Mentioned In	n/a

review active

State where review processing is currently active for a pull request.

Key	Value
Kind	Catalog Alias
Name	review active
Mentioned In	n/a

review context

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Alias
Name	review context
Mentioned In	`Architecture Intent View`

review outcome

Event indicating analysis output is ready for publication.

Key	Value
Kind	Catalog Alias

Key	Value
Name	review outcome
Mentioned In	`REQ-PRR-006`

review system

Functional group for AI and deterministic review analysis and publication.

Key	Value
Kind	Catalog Alias
Name	review system
Mentioned In	n/a

review-ready context

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Alias
Name	review-ready context
Mentioned In	n/a

runtime operations

Functional group for deployment lifecycle and secret/config controls.

Key	Value
Kind	Catalog Alias
Name	runtime operations
Mentioned In	`Architecture Intent View`

secret config

Manages credentials and secure runtime configuration.

Key	Value
Kind	Catalog Alias
Name	secret config

Key	Value
Mentioned In	n/a

secret leakage

Unintended credential disclosure in generated review findings.

Key	Value
Kind	Catalog Alias
Name	secret leakage
Mentioned In	n/a

secrets manager

Managed secret store for application credentials and config values.

Key	Value
Kind	Catalog Alias
Name	secrets manager
Mentioned In	n/a

security reviewer

Owner of security policy checks and finding triage.

Key	Value
Kind	Catalog Alias
Name	security reviewer
Mentioned In	n/a

signature valid

Condition indicating webhook HMAC verification succeeded.

Key	Value
Kind	Catalog Alias
Name	signature valid
Mentioned In	n/a

the pr review system

GitHub App review system-of-interest that ingests PR events and publishes review outcomes.

Key	Value
Kind	Catalog Alias
Name	the pr review system
Mentioned In	`REQ-PRR-001`, `REQ-PRR-002`, `REQ-PRR-003`, `REQ-PRR-004`, `REQ-PRR-005`, `REQ-PRR-006`, `REQ-PRR-007`, `REQ-PRR-008`, `REQ-PRR-009`, `REQ-PRR-010`

webhook events

Event raised when a new pull request is opened.

Key	Value
Kind	Catalog Alias
Name	webhook events
Mentioned In	n/a

webhook ingress

Validates webhook signatures and routes trusted pull request events.

Key	Value
Kind	Catalog Alias
Name	webhook ingress
Mentioned In	`Deployment View`

webhook spoofing

Forged webhook request attempting unauthorized review execution.

Key	Value
Kind	Catalog Alias
Name	webhook spoofing
Mentioned In	n/a

AV-PR-SPAM-ABUSE

High-volume event abuse causing review capacity exhaustion.

Key	Value
Kind	Catalog Attack Vector
Name	pull request spam abuse
Mentioned In	`Security View`

AV-PROMPT-INJECTION-IN-DIFF

Malicious prompt-like content embedded in pull request code changes.

Key	Value
Kind	Catalog Attack Vector
Name	prompt injection in diff
Mentioned In	`Security View`

AV-SECRET-LEAKAGE-IN-REVIEW

Unintended credential disclosure in generated review findings.

Key	Value
Kind	Catalog Attack Vector
Name	secret leakage in review output
Mentioned In	`Security View`

AV-SPOOFED-GITHUB-WEBHOOK

Forged webhook request attempting unauthorized review execution.

Key	Value
Kind	Catalog Attack Vector
Name	spoofed github webhook
Mentioned In	`Security View`

COND-WEBHOOK-SIGNATURE-VALID

Condition indicating webhook HMAC verification succeeded.

Key	Value
Kind	Catalog Condition

Key	Value
Name	webhook signature is valid
Mentioned In	n/a

DATA-CHECKRUN-SUMMARY

Aggregated review result posted as a GitHub check run.

Key	Value
Kind	Catalog Data Term
Name	check run summary
Mentioned In	`REQ-PRR-005`

DATA-DIFF-CONTEXT

Pull request file changes and metadata prepared for analysis.

Key	Value
Kind	Catalog Data Term
Name	diff context
Mentioned In	`REQ-PRR-002`

DATA-REDACTED-AUDIT-METADATA

Sanitized prompt and finding metadata retained for audit.

Key	Value
Kind	Catalog Data Term
Name	redacted audit metadata
Mentioned In	`REQ-PRR-007`

DATA-RESULT-METADATA

Metadata describing review output characteristics for audit and reporting.

Key	Value
Kind	Catalog Data Term
Name	result metadata

Key	Value
Mentioned In	n/a

DATA-WEBHOOK-SIGNATURE

Signature used to verify webhook authenticity.

Key	Value
Kind	Catalog Data Term
Name	webhook signature
Mentioned In	`Architecture Intent View`, `REQ-PRR-001`

EVT-BEDROCK-UNAVAILABLE

Event raised when Bedrock inference path is degraded.

Key	Value
Kind	Catalog Event
Name	bedrock runtime is unavailable
Mentioned In	`REQ-PRR-006`

EVT-CONTEXT-PREPARED

Event indicating context assembly completed and is ready for analysis.

Key	Value
Kind	Catalog Event
Name	diff context is prepared
Mentioned In	`REQ-PRR-003`

EVT-FINDINGS-GENERATED

Event indicating analysis output is ready for publication.

Key	Value
Kind	Catalog Event
Name	review findings are generated
Mentioned In	`REQ-PRR-005`, `REQ-PRR-009`

EVT-FINDINGS-PUBLISHED

Event indicating review output has been published to GitHub checks/comments.

Key	Value
Kind	Catalog Event
Name	review findings are published
Mentioned In	`REQ-PRR-007`

EVT-PR-OPENED

Event raised when a new pull request is opened.

Key	Value
Kind	Catalog Event
Name	pull request opened event is received
Mentioned In	`REQ-PRR-002`

EVT-PR-SYNCHRONIZE

Event raised when new commits are pushed to an open pull request.

Key	Value
Kind	Catalog Event
Name	pull request synchronize event is received
Mentioned In	`REQ-PRR-001`

EVT-RATE-LIMIT-EXCEEDED

Event raised when PR review rate guardrails are hit.

Key	Value
Kind	Catalog Event
Name	review request rate limit is exceeded
Mentioned In	`REQ-PRR-008`, `REQ-PRR-010`

FEAT-AI-REVIEW

Feature flag enabling model-assisted review.

Key	Value
Kind	Catalog Feature
Name	ai review is enabled
Mentioned In	`REQ-PRR-003`

FEAT-AUDIT-LOGGING

Feature flag enabling audit metadata persistence.

Key	Value
Kind	Catalog Feature
Name	audit logging is enabled
Mentioned In	`REQ-PRR-007`

FEAT-POLICY-CHECKS

Feature flag enabling deterministic policy checks.

Key	Value
Kind	Catalog Feature
Name	policy checks are enabled
Mentioned In	`REQ-PRR-004`

FG-PLATFORM-OPERATIONS

Functional group for deployment lifecycle and secret/config controls.

Key	Value
Kind	Catalog Functional Group
Name	platform operations
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`

FG-PR-INTEGRATION

Functional group for webhook ingress and pull request context intake.

Key	Value
Kind	Catalog Functional Group

Key	Value
Name	pr integration
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`

FG-REVIEW-INTELLIGENCE

Functional group for AI and deterministic review analysis and publication.

Key	Value
Kind	Catalog Functional Group
Name	review intelligence
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`

FU-GITHUB-WEBHOOK-INGRESS

Validates webhook signatures and routes trusted pull request events.

Key	Value
Kind	Catalog Functional Unit
Name	github webhook ingress
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Pr Opened Review Flow`, `Rate Limit Deferral Test`

FU-LAMBDA-RUNTIME-OPERATIONS

Manages runtime deployment and operational controls.

Key	Value
Kind	Catalog Functional Unit
Name	lambda runtime operations
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`

FU-POLICY-CHECKS

Evaluates deterministic repository policy checks.

Key	Value
Kind	Catalog Functional Unit
Name	policy checks
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `REQ-PRR-004`, `Policy Only Fallback Test`

FU-PR-CONTEXT-ASSEMBLY

Fetches changed files and prepares bounded review context payloads.

Key	Value
Kind	Catalog Functional Unit
Name	pr context assembly
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Pr Opened Review Flow`

FU-REVIEW-ORCHESTRATION

Coordinates Bedrock analysis with deterministic checks and fallback logic.

Key	Value
Kind	Catalog Functional Unit
Name	review orchestration
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Pr Opened Review Flow`, `Policy Only Fallback Test`

FU-REVIEW-PUBLICATION

Publishes review findings to GitHub checks and comments.

Key	Value
Kind	Catalog Functional Unit
Name	review publication
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Pr Opened Review Flow`, `Audit Redaction Test`, `Policy Only Fallback Test`, `Github Check Run Contract Test`, `Rate Limit Deferral Test`

FU-SECRETS-CONFIGURATION

Manages credentials and secure runtime configuration.

Key	Value
Kind	Catalog Functional Unit
Name	secrets and configuration
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Audit Redaction Test`

MODE-NORMAL

Default operating mode using AI plus deterministic checks.

Key	Value
Kind	Catalog Mode
Name	normal mode is active
Mentioned In	n/a

REF-AWS-BEDROCK-RUNTIME

External Bedrock runtime endpoint used for model inference.

Key	Value
Kind	Catalog Referenced Element
Name	bedrock runtime
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `REQ-PRR-003`

REF-AWS-SDK-GO

SDK dependency used for AWS runtime integrations.

Key	Value
Kind	Catalog Referenced Element
Name	aws sdk for go
Mentioned In	n/a

REF-AWS-SECRETS-MANAGER

Managed secret store for application credentials and config values.

Key	Value
Kind	Catalog Referenced Element
Name	aws secrets manager
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`

REF-GITHUB-APP-API

External GitHub API used for pull request metadata and review publication.

Key	Value
Kind	Catalog Referenced Element
Name	github app api
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`

REF-GITHUB-REST-SDK

SDK dependency used for GitHub App REST interactions.

Key	Value
Kind	Catalog Referenced Element
Name	github rest sdk
Mentioned In	n/a

REF-POLICY-RULESET

Deterministic review policy bundle used during analysis.

Key	Value
Kind	Catalog Referenced Element
Name	policy ruleset
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`

STATE-PR-REVIEW-ACTIVE

State where review processing is currently active for a pull request.

Key	Value
Kind	Catalog State
Name	pull request review is active
Mentioned In	`REQ-PRR-001`, `REQ-PRR-002`, `REQ-PRR-004`, `REQ-PRR-005`, `REQ-PRR-009`

SYS-PR-REVIEW-SYSTEM

GitHub App review system-of-interest that ingests PR events and publishes review outcomes.

Key	Value
Kind	Catalog System
Name	pr review system
Mentioned In	n/a

EM-ACTOR

A person or operational role that interacts with functional units.

Key	Value
Kind	Engineering Model Term
Name	actor
Mentioned In	`Architecture Intent View`, `Communication View`

EM-ATTACK-VECTOR

A technical misuse or attack path that targets functional, referenced, or runtime elements.

Key	Value
Kind	Engineering Model Term
Name	attack vector
Mentioned In	n/a

EM-AUTHORED-MAPPING

An explicit relationship declared in architecture inputs between authored or referenced elements.

Key	Value
Kind	Engineering Model Term

Key	Value
Name	authored mapping
Mentioned In	`Communication View`, `Security View`

EM-CODE-ELEMENT

An inferred code structure or ownership element discovered from source trees and build metadata.

Key	Value
Kind	Engineering Model Term
Name	code element
Mentioned In	n/a

EM-FUNCTIONAL-GROUP

A major authored capability area that groups related functional units.

Key	Value
Kind	Engineering Model Term
Name	functional group
Mentioned In	n/a

EM-FUNCTIONAL-UNIT

An authored working unit inside a functional group that owns specific behavior.

Key	Value
Kind	Engineering Model Term
Name	functional unit
Mentioned In	n/a

EM-INFERRED-MAPPING

A discovered relationship that links inferred runtime/code elements upward to authored design.

Key	Value
Kind	Engineering Model Term
Name	inferred mapping

Key	Value
Mentioned In	n/a

EM-REFERENCED-ELEMENT

An architecture-relevant external, platform, or third-party dependency represented by role.

Key	Value
Kind	Engineering Model Term
Name	referenced element
Mentioned In	n/a

EM-REQUIREMENT

A structured requirement statement that defines expected system behavior and maps to functional ownership.

Key	Value
Kind	Engineering Model Term
Name	requirement
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `REQ-PRR-006`

EM-RUNTIME-ELEMENT

An inferred runtime realization element discovered from infrastructure and deployment sources.

Key	Value
Kind	Engineering Model Term
Name	runtime element
Mentioned In	n/a

EM-UPWARD-LINKING

Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

Key	Value
Kind	Engineering Model Term
Name	upward linking

Key	Value
Mentioned In	n/a

EM-VERIFICATION-CHECK

An inferred or authored verification artifact that validates requirement behavior with test evidence.

Key	Value
Kind	Engineering Model Term
Name	verification check
Mentioned In	n/a

11.3. Inferred Runtime References

review_orchestrator

assembles diff context and orchestrates deterministic plus Bedrock-backed review analysis

Key	Value
Kind	Runtime lambda_function
Owner	FU-REVIEW-ORCHESTRATION
Source	examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf
Mentioned In	`Deployment View`, `Security View`

review_publisher

publishes check run summaries and inline review comments back to GitHub

Key	Value
Kind	Runtime lambda_function
Owner	FU-REVIEW-PUBLICATION
Source	examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf
Mentioned In	`Deployment View`, `Security View`

webhook_ingress

validates GitHub webhook signature and normalizes pull request event payloads

Key	Value
Kind	Runtime lambda_function
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	examples/bedrock-pr-review-github-app-sample/infra/terraform/main.tf
Mentioned In	`Deployment View`, `Security View`

11.4. Inferred Code References

aws-sdk-go-v2/secretsmanager

Inferred library_external dependency owned by FU-SECRETS-CONFIGURATION from secrets_configuration.ts.

Key	Value
Kind	Code library_external
Owner	FU-SECRETS-CONFIGURATION
Source	secrets_configuration.ts
Mentioned In	n/a

github-rest-sdk

Inferred library_external dependency owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts.

Key	Value
Kind	Code library_external
Owner	FU-PR-CONTEXT-ASSEMBLY
Source	pr_context_assembly.ts
Mentioned In	n/a

github-rest-sdk/publication-client

Inferred library_external dependency owned by FU-REVIEW-PUBLICATION from review_publication.ts.

Key	Value
Kind	Code library_external
Owner	FU-REVIEW-PUBLICATION
Source	review_publication.ts

Key	Value
Mentioned In	n/a

serde_json::json

Inferred library_external dependency owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs.

Key	Value
Kind	Code library_external
Owner	FU-REVIEW-ORCHESTRATION
Source	review_orchestrator.rs
Mentioned In	n/a

crypto/hmac

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go
Mentioned In	n/a

crypto/sha256

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go
Mentioned In	n/a

encoding/hex

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go
Mentioned In	n/a

fmt

Inferred library_stdlib dependency owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go
Mentioned In	n/a

log

Inferred library_stdlib dependency owned by FU-LAMBDA-RUNTIME-OPERATIONS from lambda_runtime_ops.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-LAMBDA-RUNTIME-OPERATIONS
Source	lambda_runtime_ops.go
Mentioned In	n/a

lambda_runtime_ops.go

wires Lambda runtime entrypoints and shared execution dependencies

Key	Value
Kind	Code source_file
Owner	FU-LAMBDA-RUNTIME-OPERATIONS
Source	lambda_runtime_ops.go
Mentioned In	`Deployment View`

policy_checks.rs

evaluates policy and style rules before review publication

Key	Value
Kind	Code source_file
Owner	FU-POLICY-CHECKS
Source	policy_checks.rs
Mentioned In	`Deployment View`

pr_context_assembly.ts

collects pull request diff context and prepares review input payloads

Key	Value
Kind	Code source_file
Owner	FU-PR-CONTEXT-ASSEMBLY
Source	pr_context_assembly.ts
Mentioned In	`Deployment View`, `Security View`

review_orchestrator.rs

coordinates deterministic and AI review steps for pull requests

Key	Value
Kind	Code source_file
Owner	FU-REVIEW-ORCHESTRATION
Source	review_orchestrator.rs
Mentioned In	n/a

review_publication.ts

publishes review comments and check-run outcomes back to GitHub

Key	Value
Kind	Code source_file
Owner	FU-REVIEW-PUBLICATION

Key	Value
Source	review_publication.ts
Mentioned In	`Deployment View`, `Security View`

secrets_configuration.ts

loads GitHub and Bedrock secret configuration for review execution

Key	Value
Kind	Code source_file
Owner	FU-SECRETS-CONFIGURATION
Source	secrets_configuration.ts
Mentioned In	`Deployment View`, `Security View`

webhook_ingress.go

validates GitHub webhook signatures and routes pull request events

Key	Value
Kind	Code source_file
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go
Mentioned In	n/a

CODE-BUILDDIFFCONTEXT

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:15.

Key	Value
Kind	Code symbol
Owner	FU-PR-CONTEXT-ASSEMBLY
Source	pr_context_assembly.ts:15
Mentioned In	n/a

CODE-FALLBACKPOLICYONLY

Inferred code symbol owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs:19.

Key	Value
Kind	Code symbol
Owner	FU-REVIEW-ORCHESTRATION
Source	review_orchestrator.rs:19
Mentioned In	n/a

CODE-FETCHCHANGEDFILES

Inferred code symbol owned by FU-PR-CONTEXT-ASSEMBLY from pr_context_assembly.ts:9.

Key	Value
Kind	Code symbol
Owner	FU-PR-CONTEXT-ASSEMBLY
Source	pr_context_assembly.ts:9
Mentioned In	n/a

CODE-GETGITHUBAPPPRIVATEKEY

Inferred code symbol owned by FU-SECRETS-CONFIGURATION from secrets_configuration.ts:9.

Key	Value
Kind	Code symbol
Owner	FU-SECRETS-CONFIGURATION
Source	secrets_configuration.ts:9
Mentioned In	n/a

CODE-NOTIFYMAINTAINERDEFERRAL

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:27.

Key	Value
Kind	Code symbol
Owner	FU-REVIEW-PUBLICATION
Source	review_publication.ts:27
Mentioned In	n/a

CODE-POLICYONLYSUMMARY

Inferred code symbol owned by FU-POLICY-CHECKS from policy_checks.rs:21.

Key	Value
Kind	Code symbol
Owner	FU-POLICY-CHECKS
Source	policy_checks.rs:21
Mentioned In	n/a

CODE-PUBLISHCHECKRUN

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:9.

Key	Value
Kind	Code symbol
Owner	FU-REVIEW-PUBLICATION
Source	review_publication.ts:9
Mentioned In	n/a

CODE-RECORDREDACTEDAUDITMETADATA

Inferred code symbol owned by FU-REVIEW-PUBLICATION from review_publication.ts:22.

Key	Value
Kind	Code symbol
Owner	FU-REVIEW-PUBLICATION
Source	review_publication.ts:22
Mentioned In	n/a

CODE-REQUESTBEDROCKREVIEW

Inferred code symbol owned by FU-REVIEW-ORCHESTRATION from review_orchestrator.rs:9.

Key	Value
Kind	Code symbol
Owner	FU-REVIEW-ORCHESTRATION

Key	Value
Source	review_orchestrator.rs:9
Mentioned In	n/a

CODE-ROUTEPULLREQUESTEVENT

Inferred code symbol owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go:25.

Key	Value
Kind	Code symbol
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go:25
Mentioned In	n/a

CODE-RUNDETERMINISTICCHECKS

Inferred code symbol owned by FU-POLICY-CHECKS from policy_checks.rs:7.

Key	Value
Kind	Code symbol
Owner	FU-POLICY-CHECKS
Source	policy_checks.rs:7
Mentioned In	n/a

CODE-VERIFYSIGNATURE

Inferred code symbol owned by FU-GITHUB-WEBHOOK-INGRESS from webhook_ingress.go:15.

Key	Value
Kind	Code symbol
Owner	FU-GITHUB-WEBHOOK-INGRESS
Source	webhook_ingress.go:15
Mentioned In	n/a

11.5. Verification References

VER-INF-E2E-PR_OPENED_REVIEW_FLOW-6BF0B5

exercises end-to-end pull request opened flow from webhook ingest to published review

Key	Value
Kind	e2e
Status	pass
Source	test-results/contract-checkrun.json (+2)
Name	Pr Opened Review Flow
Mentioned In	`Traceability View`, `Pr Opened Review Flow`

VER-INF-INTEGRATION-AUDIT_REDACTION_TEST-B108A1

checks audit evidence redaction for secrets and sensitive payload fields

Key	Value
Kind	integration
Status	pass
Source	test-results/audit-redaction-report.md (+1)
Name	Audit Redaction Test
Mentioned In	`Traceability View`, `Audit Redaction Test`

VER-INF-INTEGRATION-POLICY_ONLY_FALLBACK_TEST-76E20A

verifies policy-only fallback when Bedrock inference is unavailable

Key	Value
Kind	integration
Status	pass
Source	test-results/integration-policy-fallback.json (+1)
Name	Policy Only Fallback Test
Mentioned In	`Traceability View`, `Policy Only Fallback Test`

VER-INF-TEST-GITHUB_CHECK_RUN_CONTRACT_TEST-17DBA6

validates GitHub check-run contract shape and required review result fields

Key	Value
Kind	test
Status	not-run
Source	tests/contract/github_check_run_contract_test.go
Name	Github Check Run Contract Test
Mentioned In	`Traceability View`, `Github Check Run Contract Test`

VER-INF-UNIT-RATE_LIMIT_DEFERRAL_TEST-DC9EB2

verifies rate-limit deferral behavior for pull request review processing

Key	Value
Kind	unit
Status	pass
Source	test-results/unit-rate-limit.xml (+1)
Name	Rate Limit Deferral Test
Mentioned In	`Traceability View`, `Rate Limit Deferral Test`